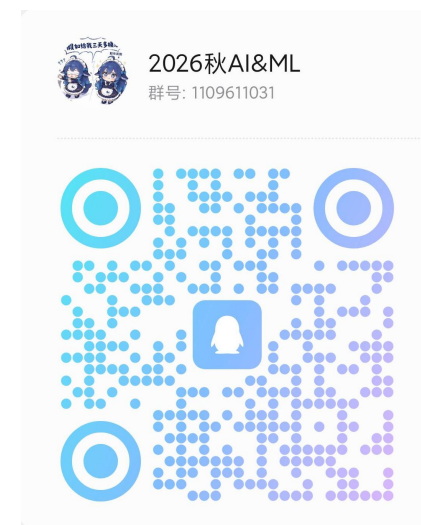


人工智能与机器学习基础

Lecture 02: Linear Regression

冯文杰

人工智能与数据科学学院
中国科学技术大学



示例：到达时间预测

- The ETA (Estimated Time of Arrival) Problem



Time of Wait

ETA

Time on the Road

ETA



Time of Wait



Time on the Road

示例：到达时间预测

- The ETA (Estimated Time of Arrival) Problem



Time of Wait

ETA

Time on the Road

ETA



Time of Wait



Time on the Road

How to solve this problem?



机器学习

- 将数据转换为模型的过程
- 使用该模型进行推断与预测
- 评估与迭代以提升泛化能力

“Learning is any process by which a system improves performance from experience.”

(学习是系统从经验中提高性能的任何过程)

——Herbert Simon (赫伯特·西蒙/司马贺)

图灵奖、诺贝尔奖得主，计算机科学家

We define *machine learning* as a set of methods that can automatically **detect patterns in data**, and then use the uncovered patterns to **predict future data**, or to **perform other kinds of decision making under uncertainty** (such as planning how to win a game).

— 《Machine Learning: A probabilistic perspective》



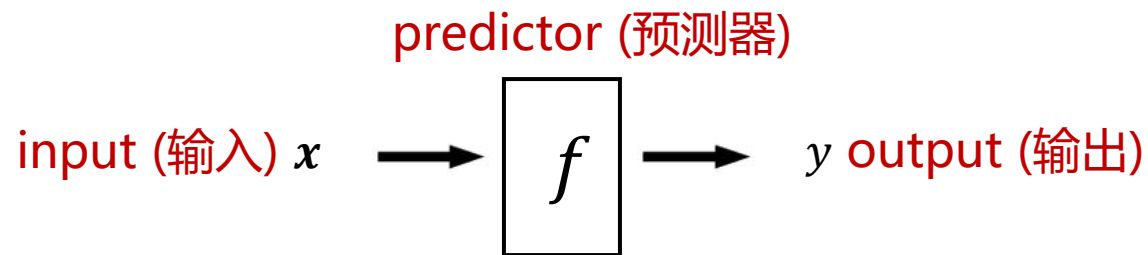
Learning from Data

- 人工智能近期成功的主要驱动力
- 将复杂性从“代码”转移到“数据”
- 需要泛化能力

机器学习要素：输入、模型（假设）、推理

- **Given (input)**
 - 样本的收集 (文档, 测量, 基因序列, ...)
 - 样本的编码 (字符串, 图片, 数据库记录, ...)
- **Hypothesis (model)**
 - 模型应当描述样本自身和样本之间的关系
 - 模型应当刻画来自同一来源/分布的样本
 - 模型可以做什么?
 - 图像分类: 细胞图像 → 是否为癌症
 - 文本分类: 文档 → 是否描述相似主题
 - 自动驾驶: 周围环境的激光雷达图像 → 汽车的路径规划
 - ...
- **Inference (estimate, predict)**
 - 使用样本输入学习 (估计) 模型的参数
 - 根据 (新) 样本输入预测输出

机器学习

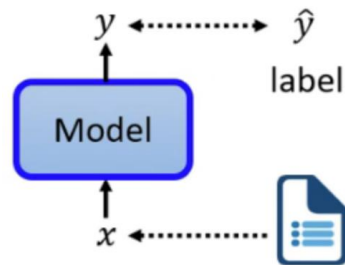


- x : 通常是一个向量 (例如: 一个句子/ 图片)
- y : 取决于任务类型 (task-dependent)

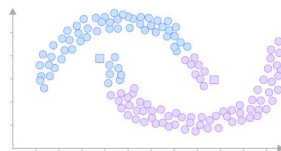
机器学习模型本质上是将输入映射为输出的数学函数

机器学习基本方法

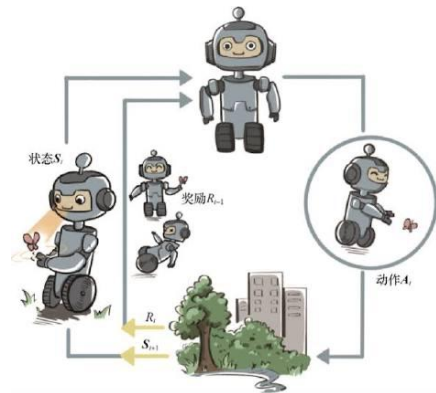
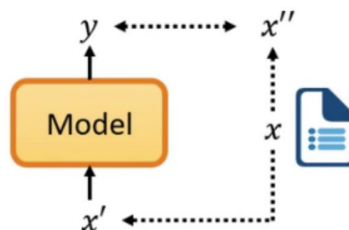
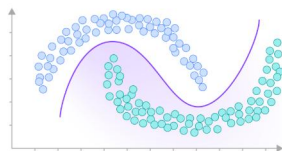
- 有监督学习 (supervised learning)
 - 数据集中的样本带有标签，有明确目标
 - 在样本-标签对数据中，找到样本到标签的最佳映射
 - E.g., 回归和分类
- 无监督学习 (unsupervised learning)
 - 数据集中的样本没有标签，没有明确目标
 - 在**没有标签**的数据集中，让模型自主发现数据内部隐藏的结构、规律或模式
 - E.g., 聚类、降维、密度估计、关联规则挖掘
- 强化学习 (reinforcement learning)
 - 智慧决策的过程，通过过程模拟和观察来不断学习、提高决策能力
 - 让一个智能体在与环境的不断**交互**试错中，根据奖励或惩罚信号，自学出一套最优的行动策略，以最大化累计收益
 - E.g., AlphaGo (蒙特卡洛树搜索)
- 自监督学习 (self-supervised learning)
 - 通过重建数据 (自己监督自己) 进行自我学习
 - 大语言模型的预训练是当前自监督学习最成功的应用



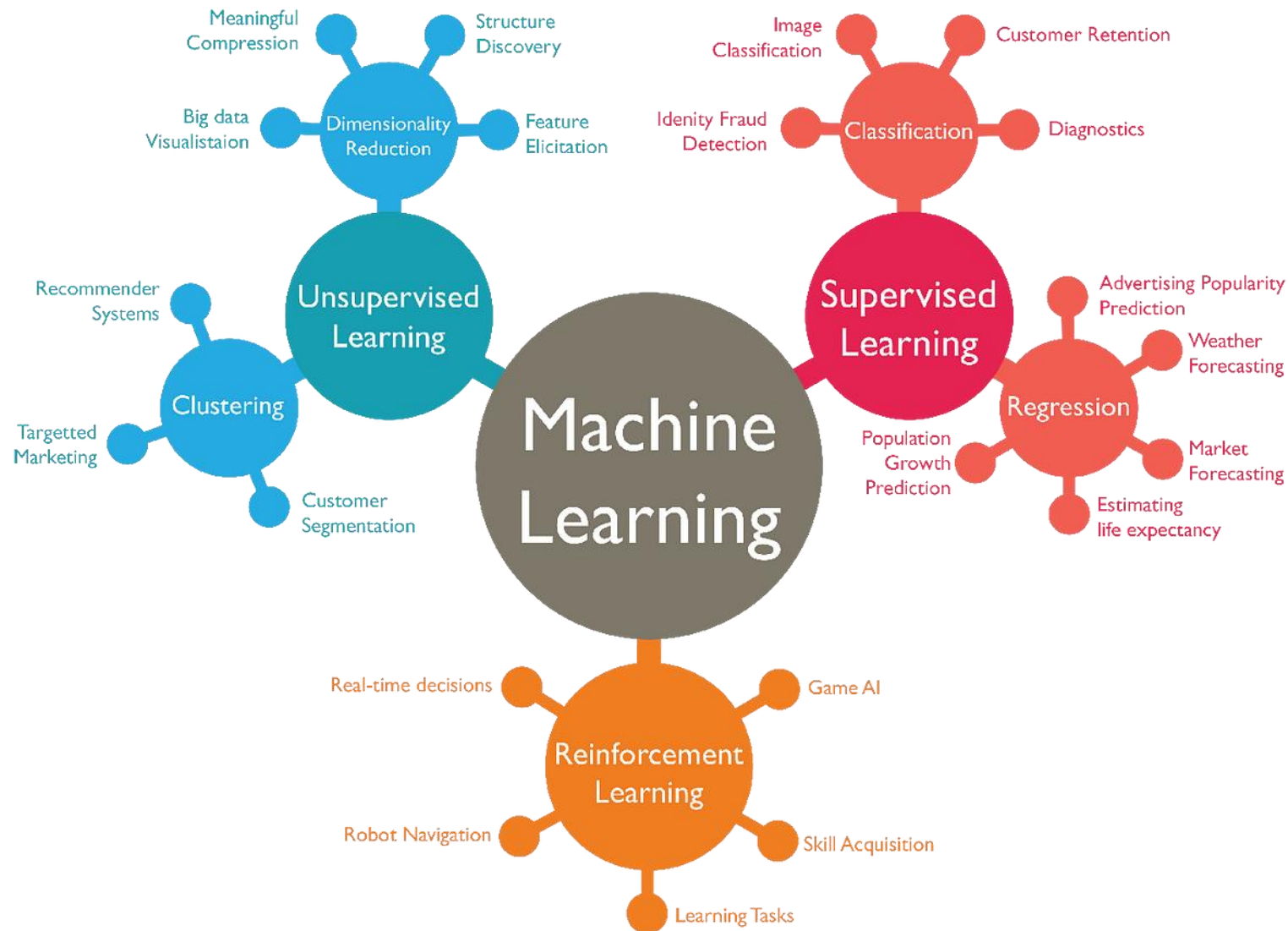
Unsupervised clustering
Centroid



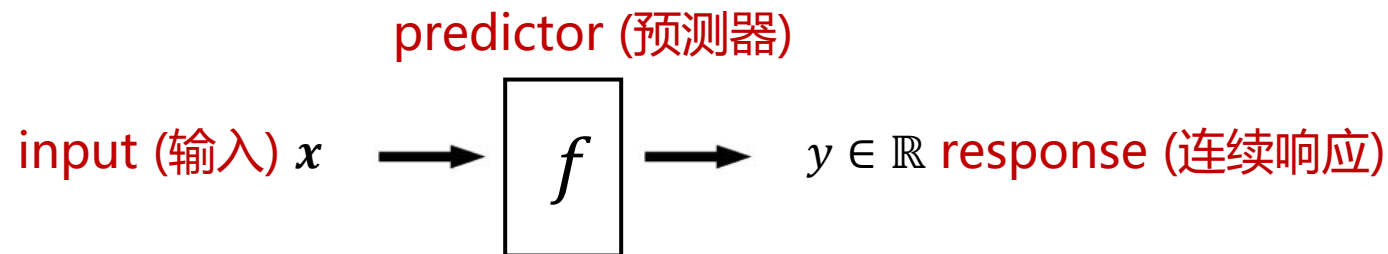
Ideal decision boundary



机器学习基本方法



监督学习：回归问题 Regression



贫困地区预测: 卫星图像 $\rightarrow$ 资产财富指数

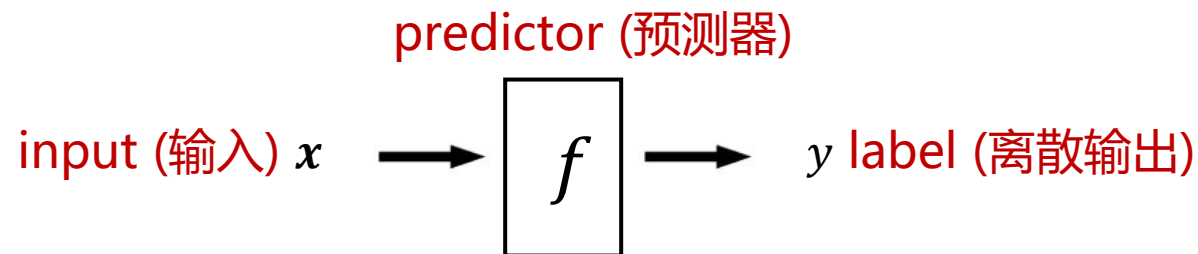


住房价格预测: 房屋信息 $\rightarrow$ 价格



到达时间预测: 目的地、天气、出发时间 $\rightarrow$ 预计到达时间

监督学习：分类问题 Classification



二分类任务



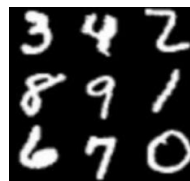
欺诈检测: 信用卡交易 $\rightarrow$ 欺诈或非欺诈



有毒评论检测: 在线评论 $\rightarrow$ 有毒或无毒

$$y \in \{+1, -1\}$$

多分类任务



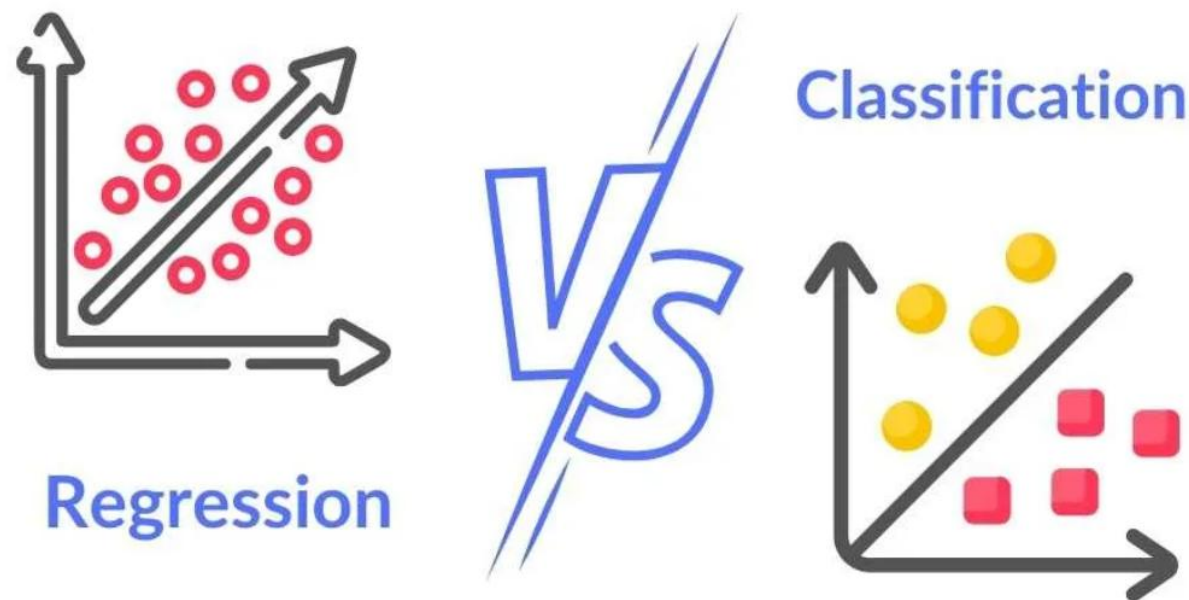
手写数字识别: 数字图像 $\rightarrow 0, 1, 2, \dots$

$$y \in \{0, 1, \dots, 9\}$$

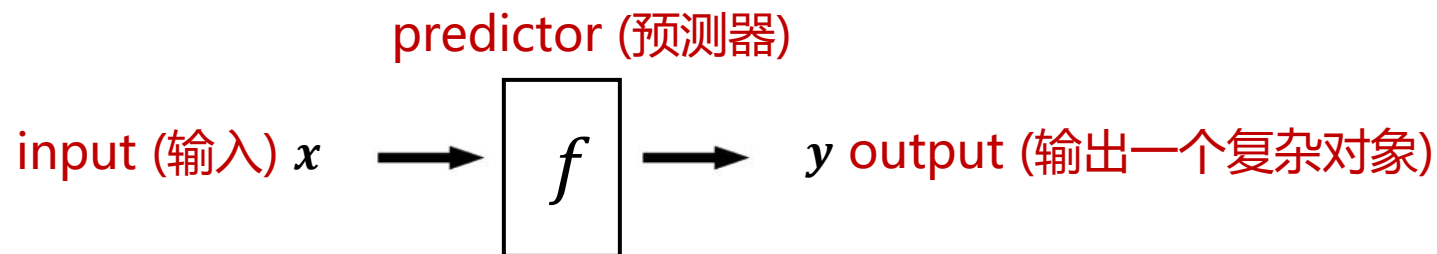
$$y \in \{1, \dots, K\}$$

监督学习：回归 vs 分类

- 回归与分类的关键区别是什么？
 - 回归的输出是连续的数值（如：价格、到达时间）
 - 分类的输出是离散的类别标签（如：欺诈/非欺诈、0~9）



监督学习：结构化预测 Structured Prediction



机器翻译: 英语句子 $\rightarrow$ 中文句子



对话: 对话历史 $\rightarrow$ 下一句话



图像描述生成: 图像 $\rightarrow$ 描述图像的句子



图像分割: 图像 $\rightarrow$ 分割结果

谷神星发现历程

- 1766年，提丢斯-波得定律 (Titius-Bode law) 预测了火星与木星之间存在一颗行星。
- 1801年1月1日，天文学家皮亚齐(Piazzi)意外发现了谷神星，但在观测41天后便失去了它的踪迹。



Johann Daniel Titius



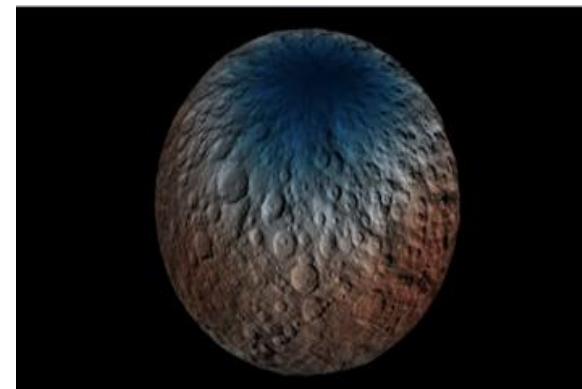
Johann Elert Bode



Giuseppe Piazzi

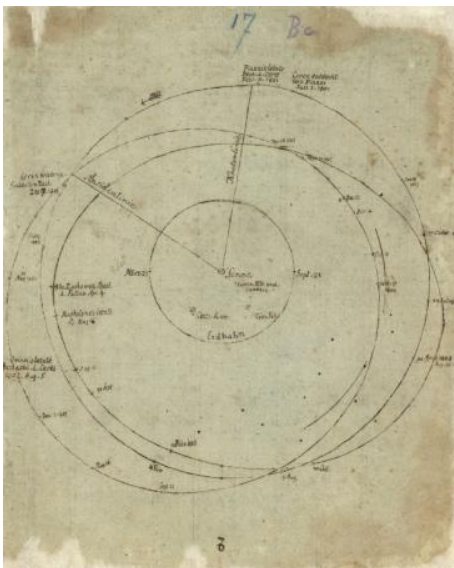
Time	Right ascension	Declination
Jan 01, 20:43:17.8	50.91	15.24
Jan 02, 20:39:04.6	50.84	15.30
...	...	...
Feb 11, 18:11:58.2	53.51	18.43

When and where will Ceres be observed again?



谷神星发现历程

- 1801年9月，24岁的卡尔·弗里德里希·高斯发明了一种新的轨道计算方法——**最小二乘法**。
- 仅凭皮亚齐41天的观测数据，高斯精确计算出了谷神星的轨道，并预测了它何时何地将在夜空中再次出现。

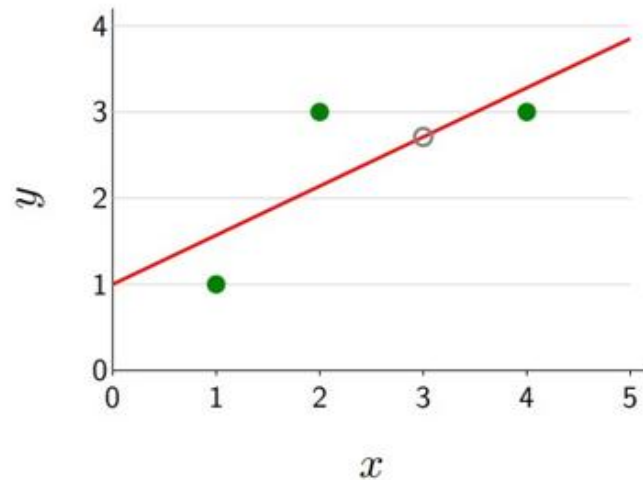
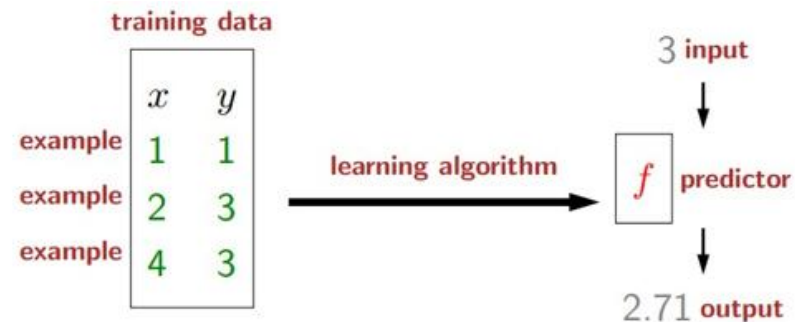


Method:
least squares linear regression
(**最小二乘回归**)

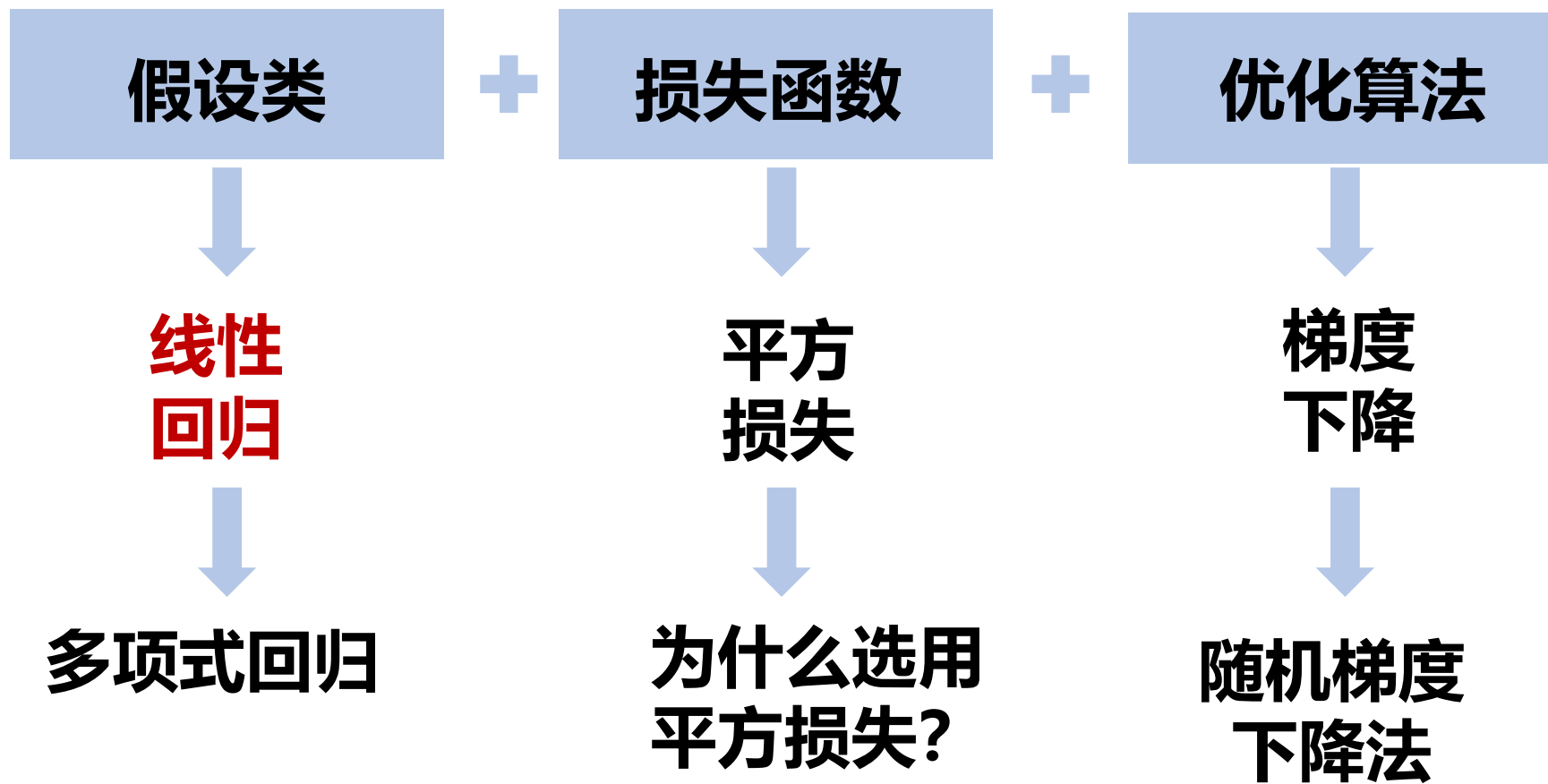
线性回归 Linear Regression: 框架

• 模型设计

- 假设类 (Hypothesis Class)
 - 使用什么样的模型 (函数形式) 来拟合数据?
- 损失函数 (Loss Function)
 - 什么样的模型被认为是“好的”?
- 优化算法 (Optimization Algorithm)
 - 我们如何找到最优的模型?



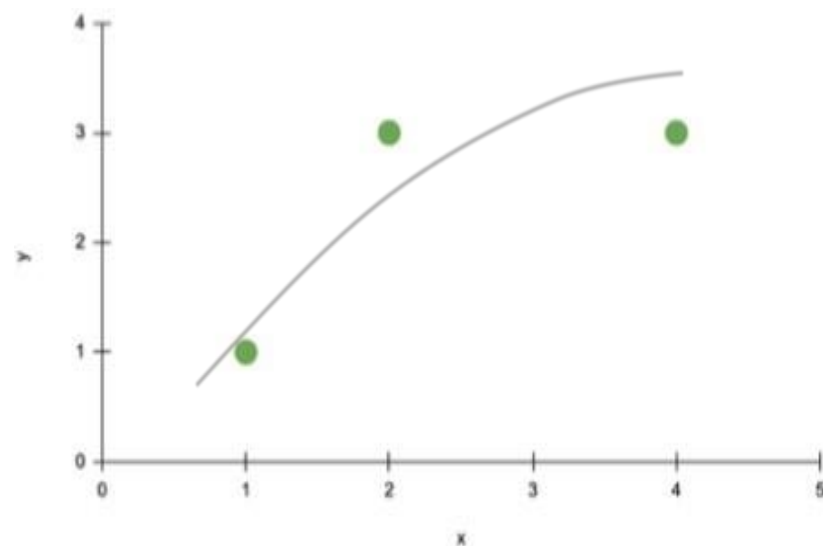
线性回归 Linear Regression: 框架



概念：假设类

- **模型**：我们对世界运行方式的假设
- **假设**： x 与 y 之间的预期关系
- **回归模型**：

- $y = f(x) + \epsilon$, $\mathbb{E}(\epsilon) = 0$
- y ：真实值
- $\hat{y} = f(x)$ ：预测值
- ϵ ：误差 $\epsilon = y - \hat{y}$



- **假设类**：所有候选模型的集合（即从输入 x 到输出 y 的映射函数 f ）
 - 注意：此时模型尚未涉及任何“线性”假设。

概念：假设类

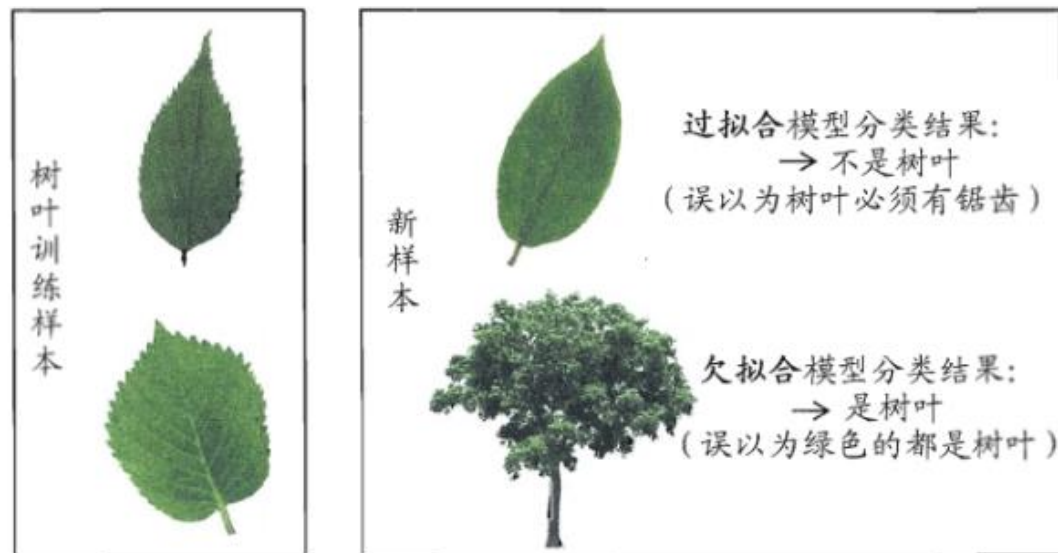
- **学习过程**：从假设类 H 中选择**最优假设** h^* ，使其在训练数据上的风险（Risk）最小化
- **三种风险 Risks**
 - 经验风险（*Empirical Risk*）
 - 模型在当前训练数据集上的平均预测误差（或损失）。
 - 期望风险（*Expected Risk*）
 - 模型在所有可能的、未见过的数据样本上的期望预测误差。
 - 结构风险（*Structural Risk*）
 - 通过限制模型复杂度来防止过拟合。

训练集 vs 验证集 vs 测试集

- 训练集
 - 用来学习模型参数;
 - 模型通过训练集上的损失函数, 用优化算法更新参数。
- 验证集
 - 用来调超参数、选模型、早停;
 - 不参与参数更新, 用来判断模型是否过拟合、欠拟合。
- 测试集
 - 用来估计模型在未见数据上的真实泛化性能;
 - 只在模型完全训练好、超参数选好之后使用。

欠拟合 vs 过拟合

- 欠拟合 (underfitting)：指模型没有充分学到训练数据中的规律，导致它在训练集和测试集上的表现都很差。
- 过拟合 (overfitting)：模型在训练集上表现很好（经验风险很低），但在测试集/泛化场景上表现显著变差的现象。模型不仅学到了数据中的真实规律，还拟合了训练数据中的噪声。

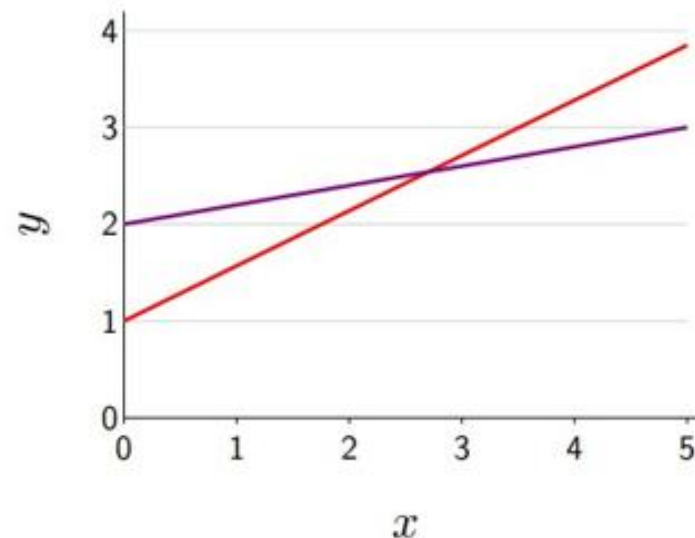


概念：线性回归模型

$$f(x) = 1 + 0.57x$$

$$f(x) = 2 + 0.2x$$

$$f(x) = w_1 + w_2x$$



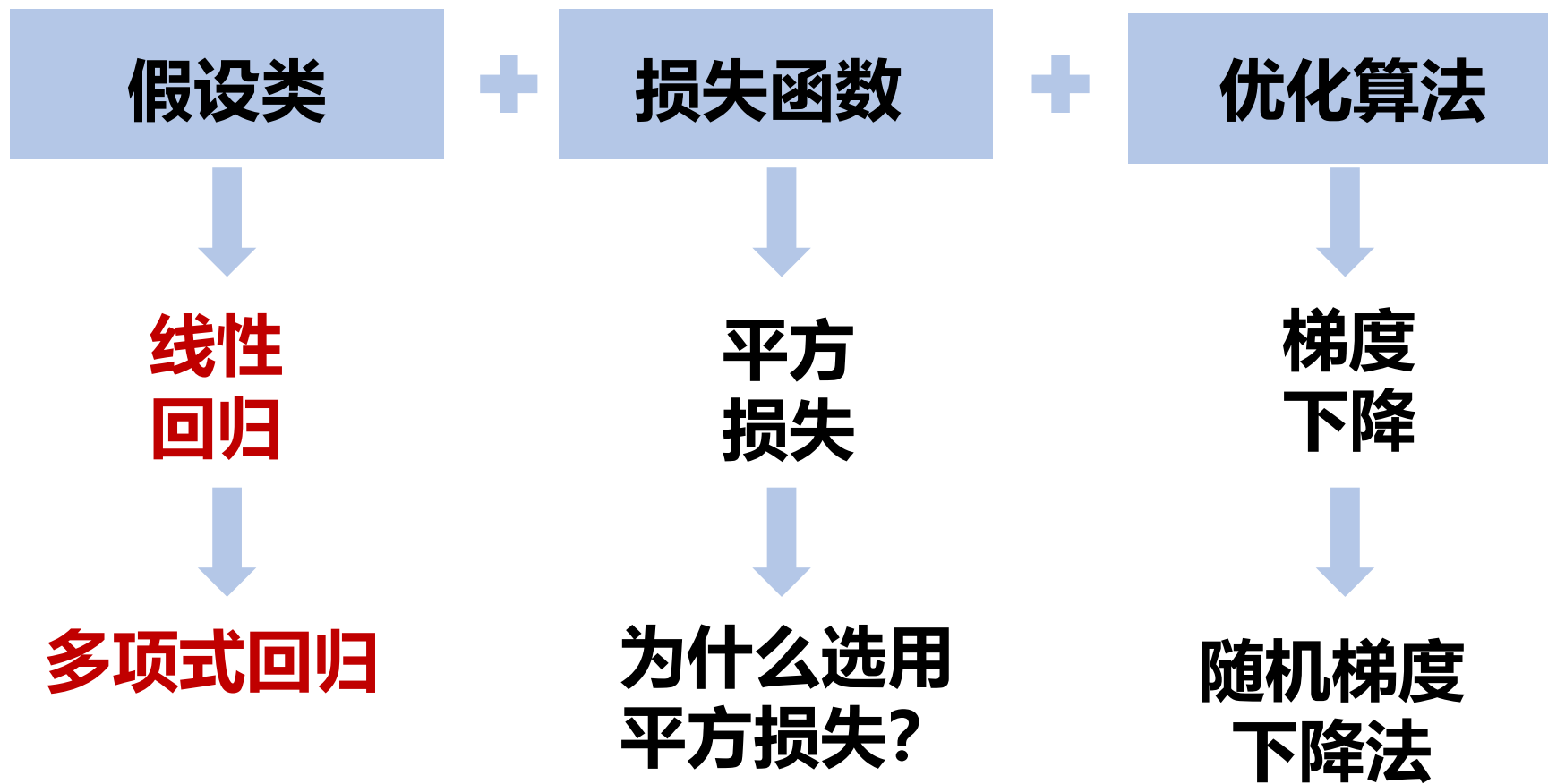
模型参数/参数向量: $\mathbf{w} = [w_1, w_2]$

特征提取器: $\phi(x) = [1, x]$ (特征向量)

模型表示: $f_{\mathbf{w}}(x) = \mathbf{w} \cdot \phi(x)$

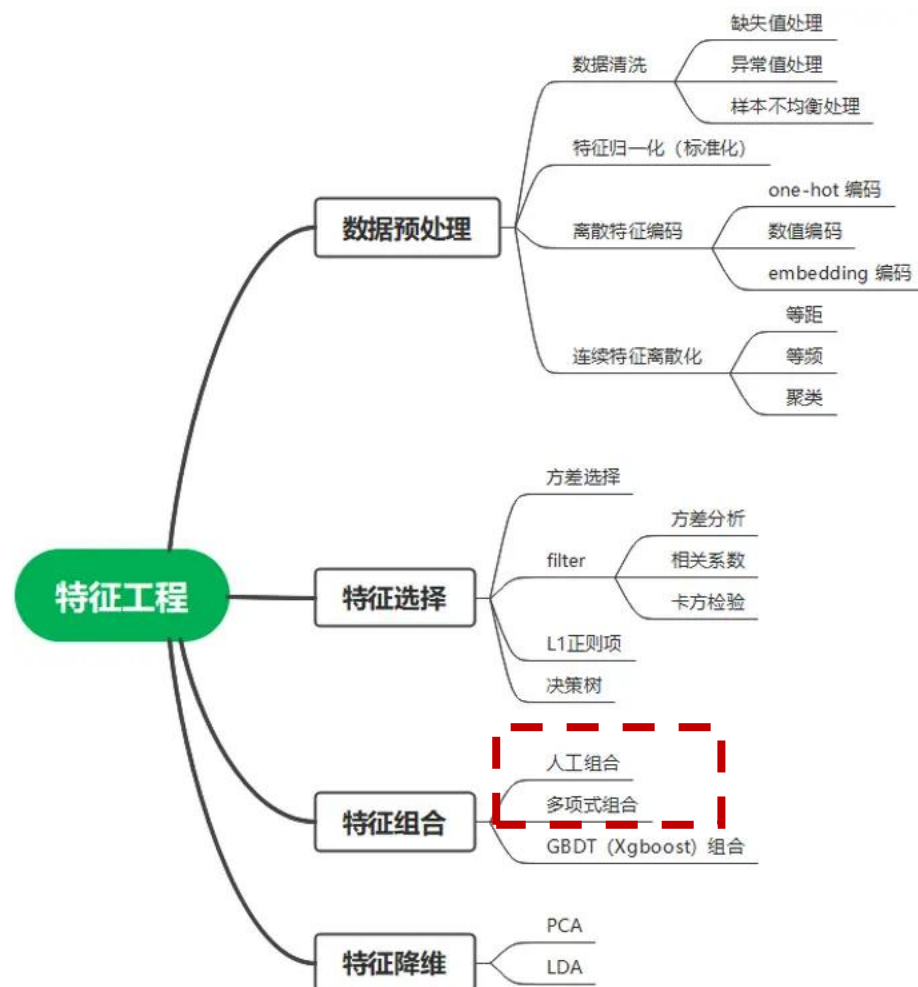
假设类: $\mathcal{F} = \{f_{\mathbf{w}}: \mathbf{w} \in \mathbb{R}^2\}$

线性回归：框架

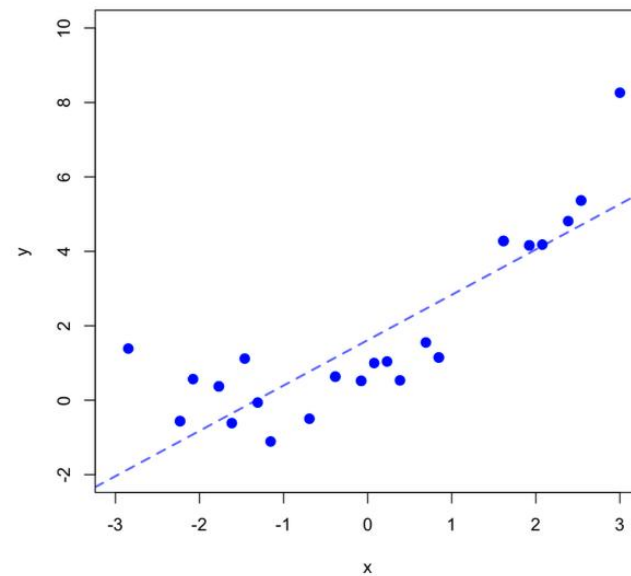
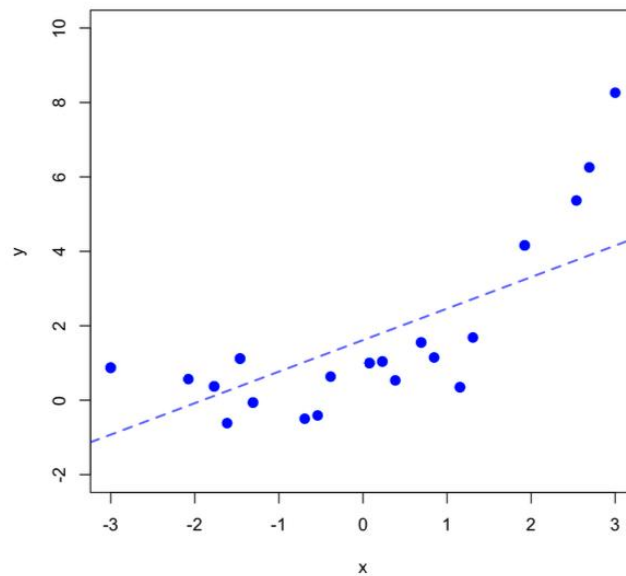
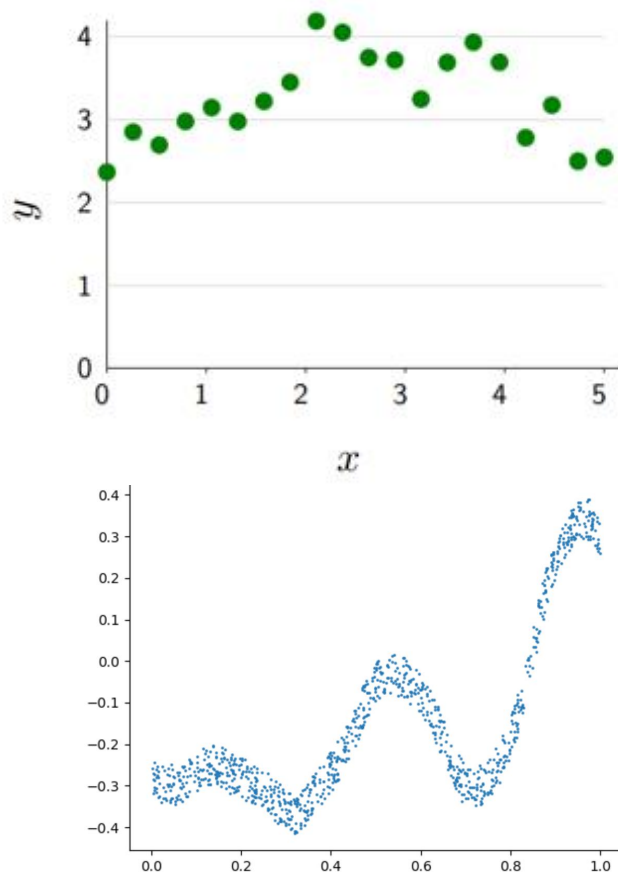


特征工程：非线性特征

- 利用线性模型的框架，获得功能强大得多的非线性预测器。

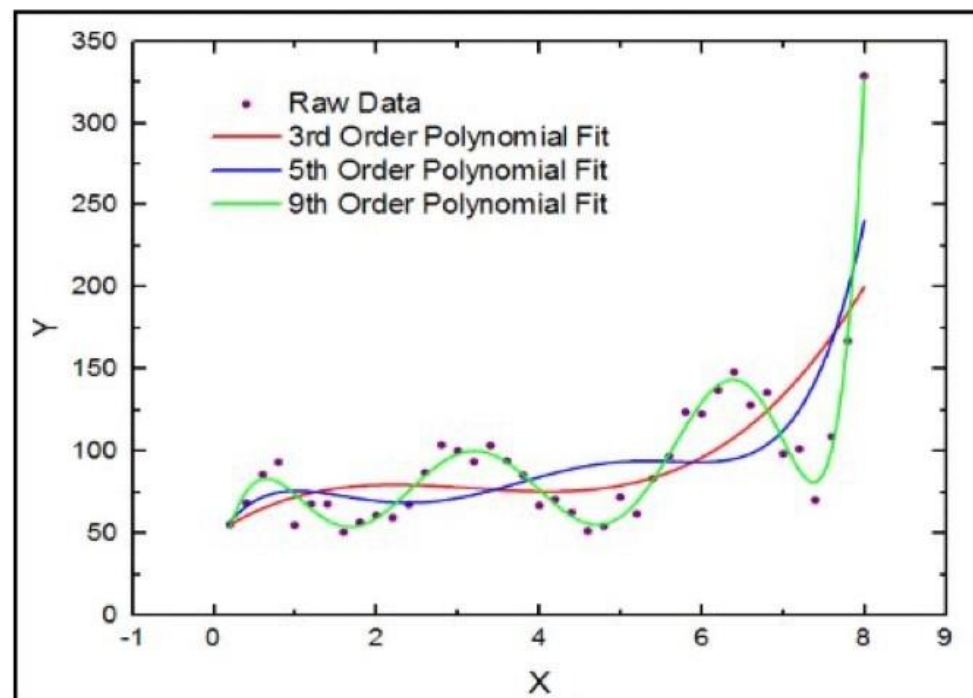
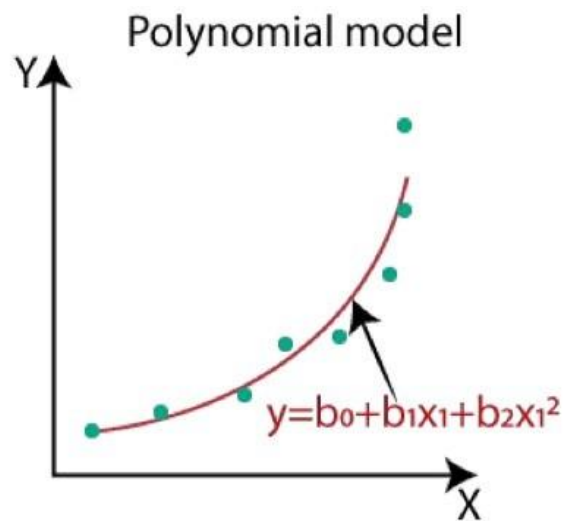
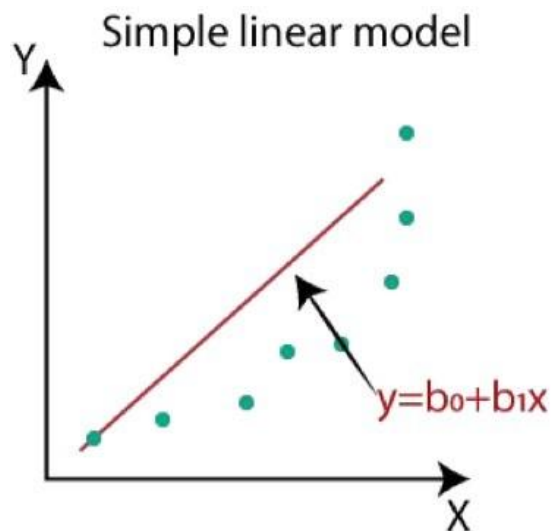


更复杂的数据



如何拟合一个非线性的预测器？

线性回归 or 多项式回归?



多项式特征

$$\phi(x) = [1, x, x^2]$$

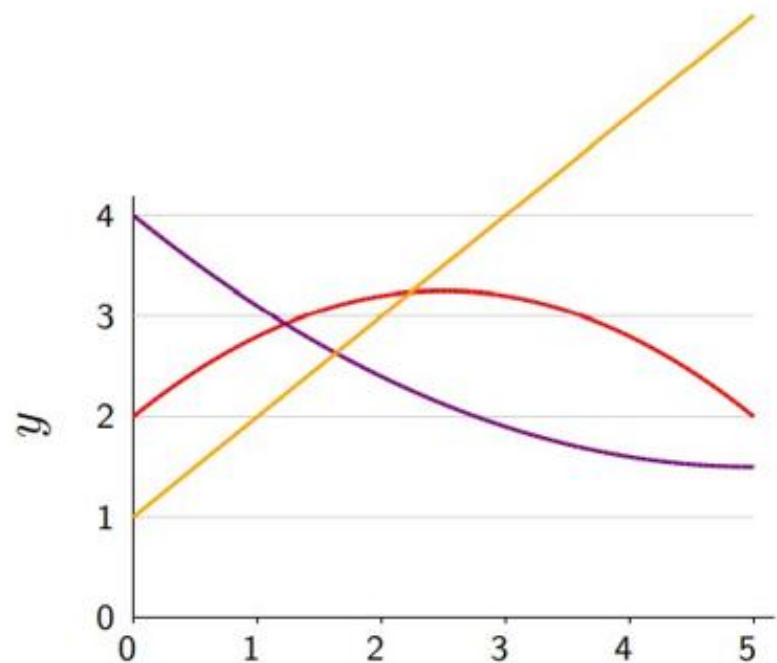
Example: $\phi(3) = [1, 3, 9]$

$$f(x) = [2, 1, -0.2] \cdot \phi(x)$$

$$f(x) = [4, -1, 0.1] \cdot \phi(x)$$

$$f(x) = [1, 1, 0] \cdot \phi(x)$$

$$\mathcal{F} = \{f_{\mathbf{w}}(x) = \mathbf{w} \cdot \phi(x) : \mathbf{w} \in \mathbb{R}^3\}$$



改变 $\phi(x)$ 构造非线性

分段常数特征

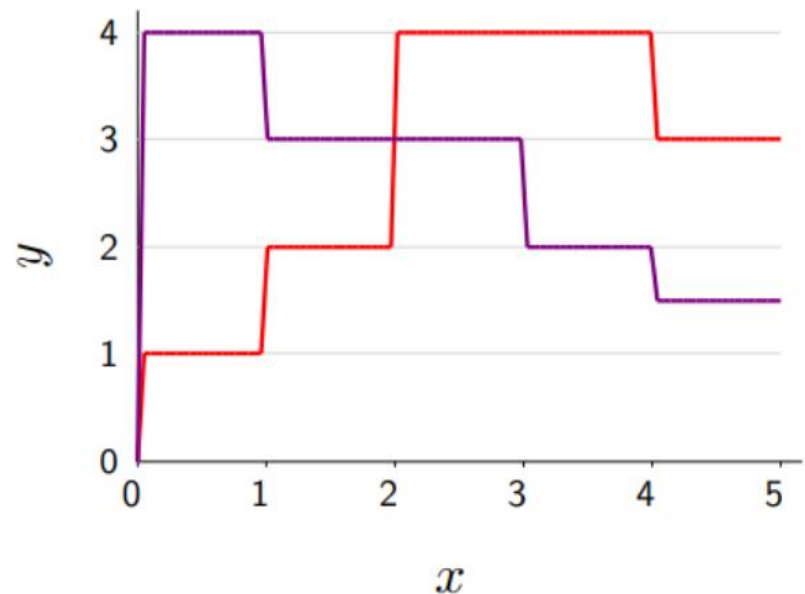
$$\phi(x) = [1[0 < x \leq 1], 1[1 < x \leq 2], 1[2 < x \leq 3], 1[3 < x \leq 4], 1[4 < x \leq 5]]$$

Example: $\phi(2.3) = [0, 0, 1, 0, 0]$

$$f(x) = [1, 2, 4, 4, 3] \cdot \phi(x)$$

$$f(x) = [4, 3, 3, 2, 1.5] \cdot \phi(x)$$

$$\mathcal{F} = \{f_{\mathbf{w}}(x) = \mathbf{w} \cdot \phi(x) : \mathbf{w} \in \mathbb{R}^5\}$$



通过划分输入空间来构建表达能力强的非线性预测器

具有周期性结构的特征

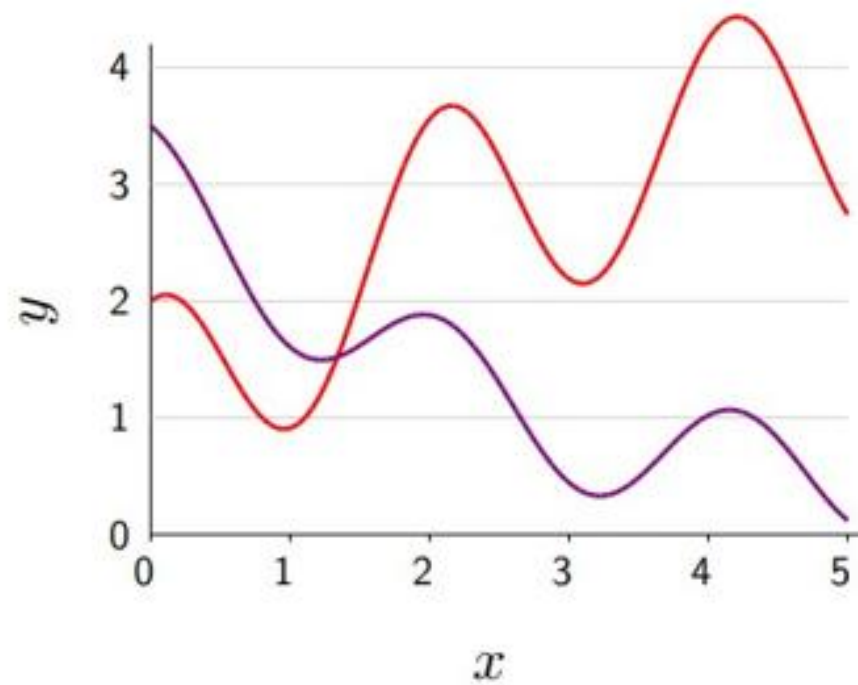
$$\phi(x) = [1, x, x^2, \cos(3x)]$$

Example: $\phi(2) = [1, 2, 4, 0.96]$

$$f(x) = [1, 1, -0.1, 1] \cdot \phi(x)$$

$$f(x) = [3, -1, 0.1, 0.5] \cdot \phi(x)$$

$$\mathcal{F} = \{f_{\mathbf{w}}(x) = \mathbf{w} \cdot \phi(x) : \mathbf{w} \in \mathbb{R}^4\}$$



为何称为线性模型

- 线性体现在哪里？

$$f_{\mathbf{w}}(x) = \mathbf{w} \cdot \phi(x)$$

- **分数 $\mathbf{w} \cdot \phi(x)$ 对于 $\mathbf{w}$ 、 $\phi(x)$ 线性，对于 x 不一定线性**
- $f(x) = w_0 + w_1 x$ 线性模型
- $f(x) = w_0 + w_1 x + w_1 x^2 = w_0 + w_1 (x + x^2)$ 线性模型
- $f(x) = w_0 + w_1 (w_2 + w_3 x)$ 非线性模型（对参数 **w**）



Key idea: non-linearity

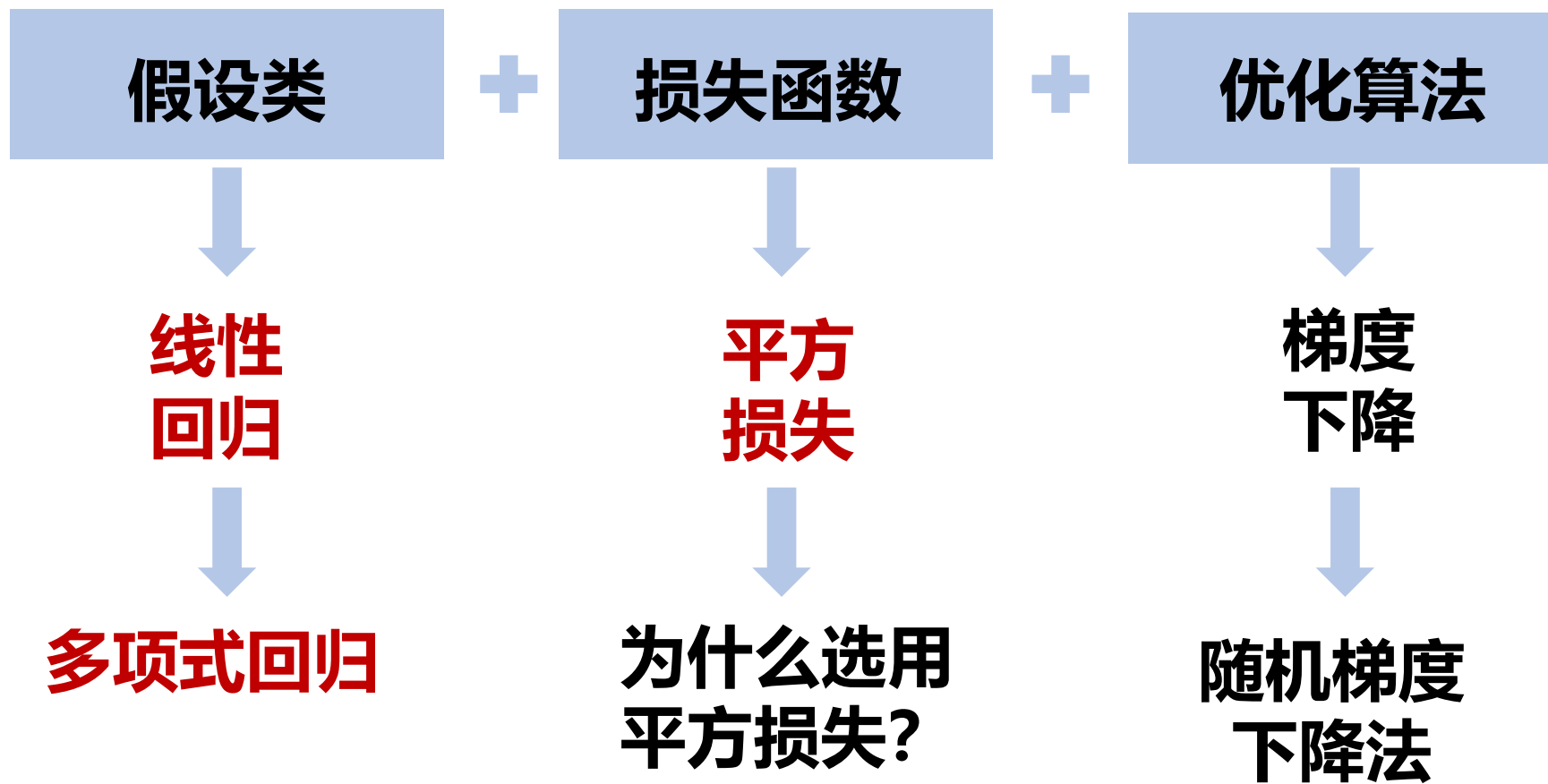
- Expressivity: score $\mathbf{w} \cdot \phi(x)$ can be a **non-linear** function of x
- Efficiency: score $\mathbf{w} \cdot \phi(x)$ always a **linear** function of $\mathbf{w}$

总结

- $f_{\mathbf{w}}(x) = \mathbf{w} \cdot \phi(x)$
 - 对于 $\mathbf{w}$ 、 $\phi(x)$ 线性
 - 对于 x 不一定线性
- 线性框架下的非线性预测
- 回归：非线性预测器
- 分类：非线性决策边界
- 非线性特征的类型：多项式特征、分段常数特征等



线性回归：框架

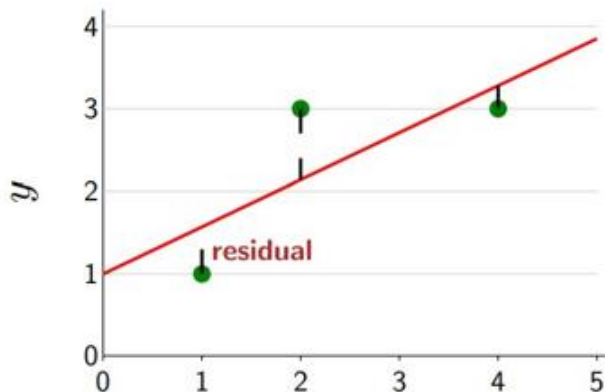


损失函数

- 平方损失 (Squared Loss) : $Loss(x, y, \mathbf{w}) = (f_{\mathbf{w}}(x) - y)^2$
- 均方误差 (Mean Squared Error) : $MSE = \frac{1}{N} \sum_{i=1}^N (f_{\mathbf{w}}(x) - y)^2$



训练损失TrainLoss



为什么是平方损失而不是绝对偏差？

- 训练数据集: $\{(1,1), (2,3), (4,3)\}$

- 参数 $\mathbf{w}$: $[1, 0.57]$; $\phi(x)[1, x]$:

- $\text{TrainLoss} = \frac{1}{3} (([1, 0.57] \cdot [1, 1] - 1)^2 + ([1, 0.57] \cdot [1, 2] - 3)^2 + ([1, 0.57] \cdot [1, 4] - 3)^2)$
 ≈ 0.38

$$\text{Loss}(1, 1, [1, 0.57]) = ([1, 0.57] \cdot [1, 1] - 1)^2 = 0.32$$

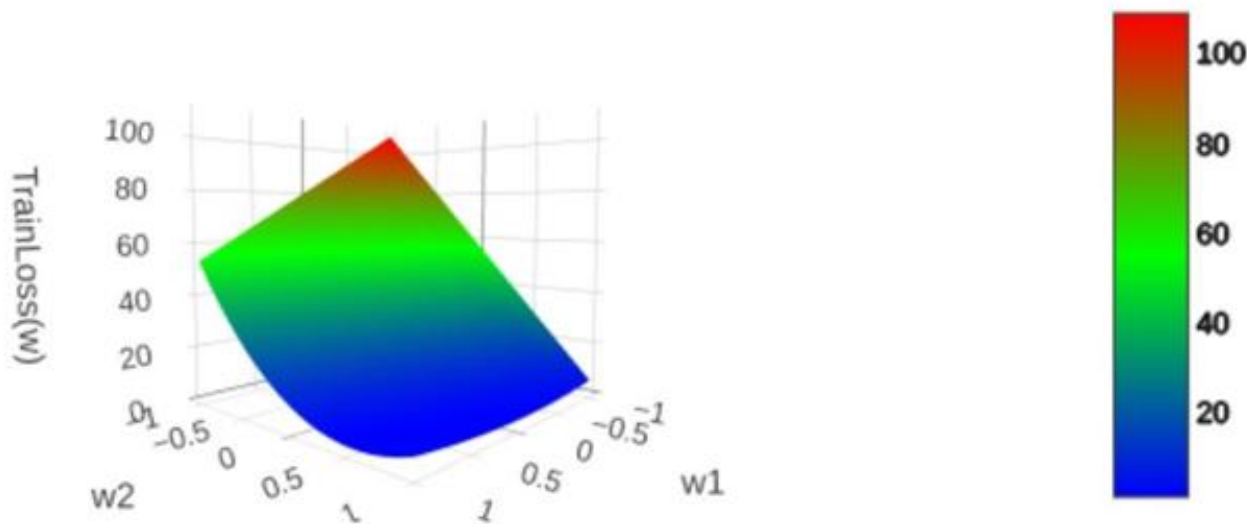
$$\text{Loss}(2, 3, [1, 0.57]) = ([1, 0.57] \cdot [1, 2] - 3)^2 = 0.74$$

$$\text{Loss}(4, 3, [1, 0.57]) = ([1, 0.57] \cdot [1, 4] - 3)^2 = 0.08$$

损失函数

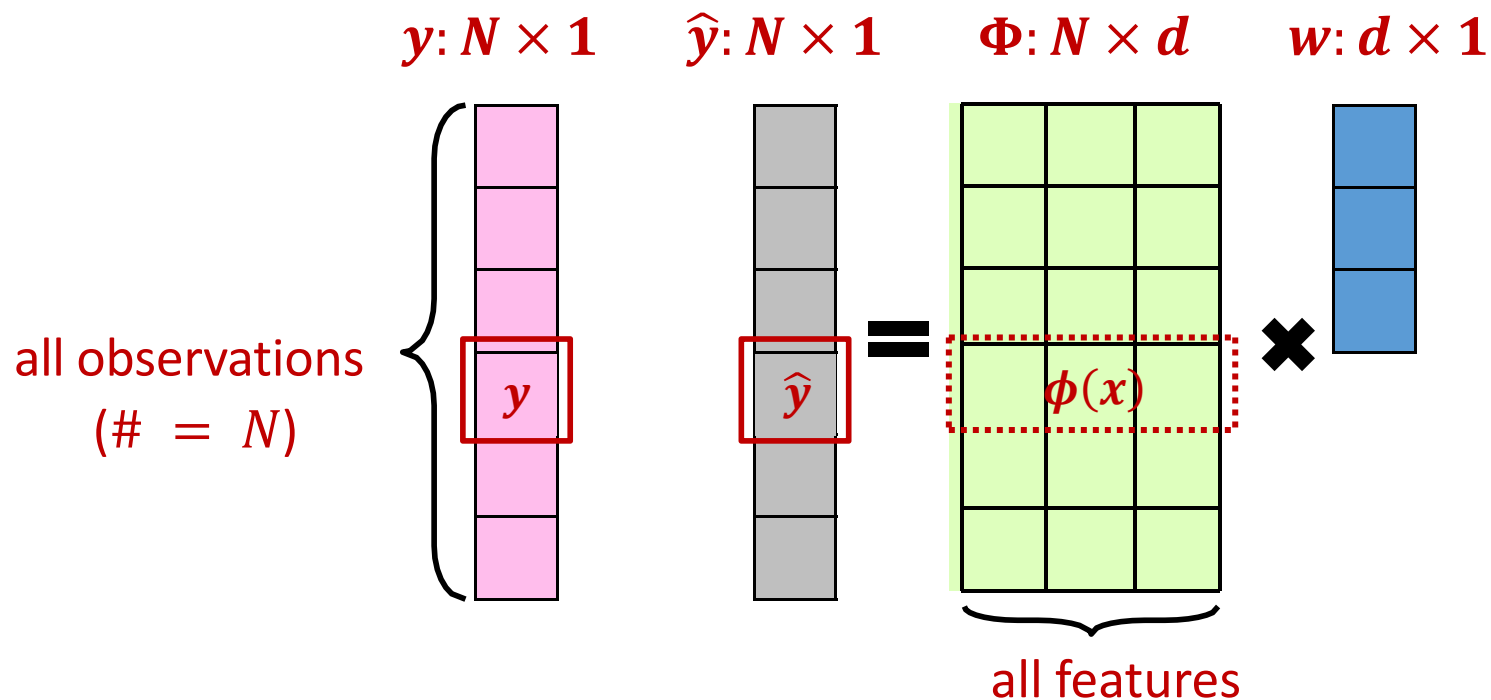
- 在所有 $\mathbf{w}$ 上最小化损失函数

$$\min_{\mathbf{w}} \text{TrainLoss}(\mathbf{w}) = \min_{\mathbf{w}} \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{(x,y) \in \mathcal{D}_{\text{train}}} (f_{\mathbf{w}}(x) - y)^2$$

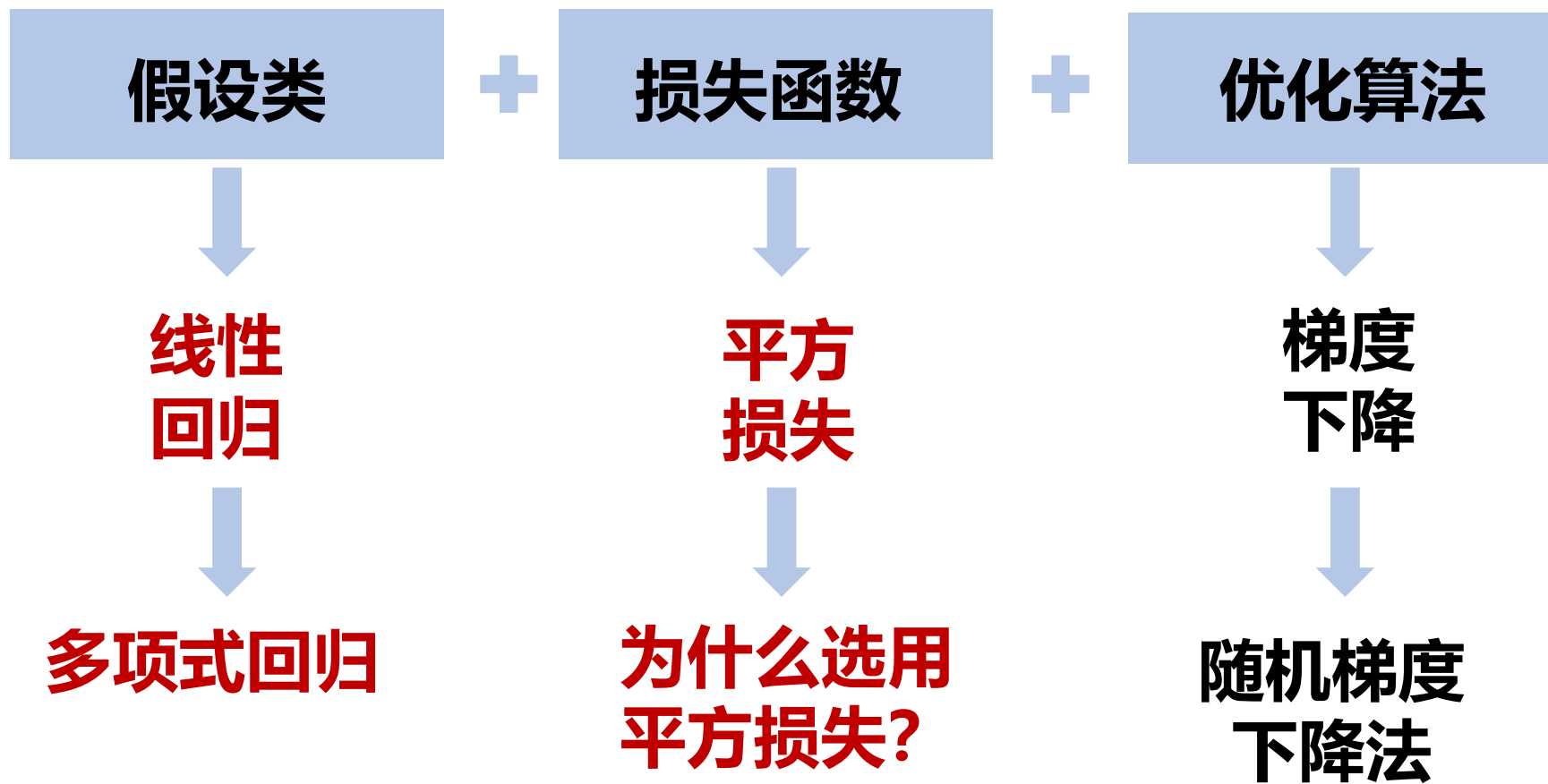


损失函数的矩阵形式

$$\begin{aligned}\text{TrainLoss}(\mathbf{w}) &= \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{(x,y) \in \mathcal{D}_{\text{train}}} (y - f_{\mathbf{w}}(x))^2 \\ &= \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{(x,y) \in \mathcal{D}_{\text{train}}} (y - \mathbf{w} \cdot \phi(x))^2 \\ &= \frac{1}{|\mathcal{D}_{\text{train}}|} (\mathbf{y} - \Phi \mathbf{w})^T (\mathbf{y} - \Phi \mathbf{w})\end{aligned}$$



线性回归：框架



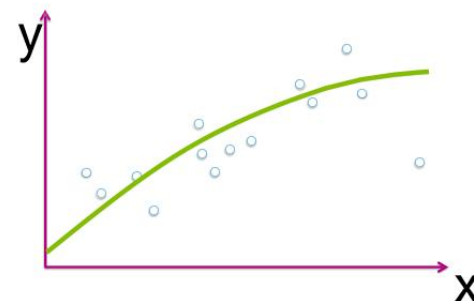
平方损失 vs 绝对损失

- **平方损失在数学上更易于优化**
 - 平方损失是一个光滑的凸函数，处处连续且可导。
- **平方损失对较大的误差施加更重的惩罚**
 - 与绝对值损失相比，平方损失的惩罚呈二次增长。
- **在统计学中，平方损失与正态分布相关**
 - 如果我们假设线性回归中的噪声服从高斯分布，那么平方损失就会自然地作为最大似然估计（MLE）的结果而出现。

最小化平方误差的概率学解释

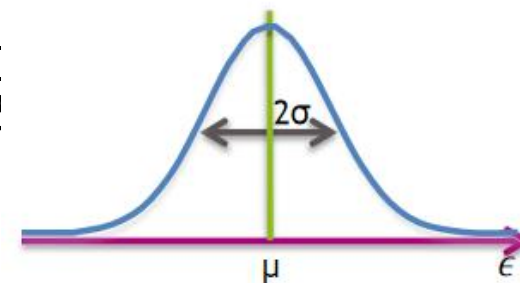
- 假设存在误差项（如噪声）：

$$y = f(x) + \epsilon; \quad \mathbb{E}(\epsilon) = 0$$



- 进一步假设： $\epsilon \sim \mathcal{N}(\mu, \sigma^2)$

$$p(\epsilon) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(\epsilon-\mu)^2}{2\sigma^2}}$$



$$y = \mathbf{w} \cdot \phi(x) + \epsilon$$
$$p(y|x; \mathbf{w}) \sim \mathcal{N}(\mathbf{w} \cdot \phi(x), \sigma^2)$$

最小化平方误差的概率学解释

- 最大似然函数：

$$\operatorname{argmax}_{\mathbf{w}} p(y|x, \mathbf{w}) = \operatorname{argmax}_{\mathbf{w}} \prod_{(x,y) \in \mathcal{D}} p(y|x, \mathbf{w})$$

- 对数似然函数：

$$\operatorname{argmax}_{\mathbf{w}} \ln p(y|x, \mathbf{w}) = \operatorname{argmax}_{\mathbf{w}} \ln \prod_{(x,y) \in \mathcal{D}} p(y|x, \mathbf{w})$$

$$= \operatorname{argmax}_{\mathbf{w}} \sum_{(x,y) \in \mathcal{D}} \ln \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(y - \mathbf{w} \cdot \phi(x))^2}{2\sigma^2}}$$

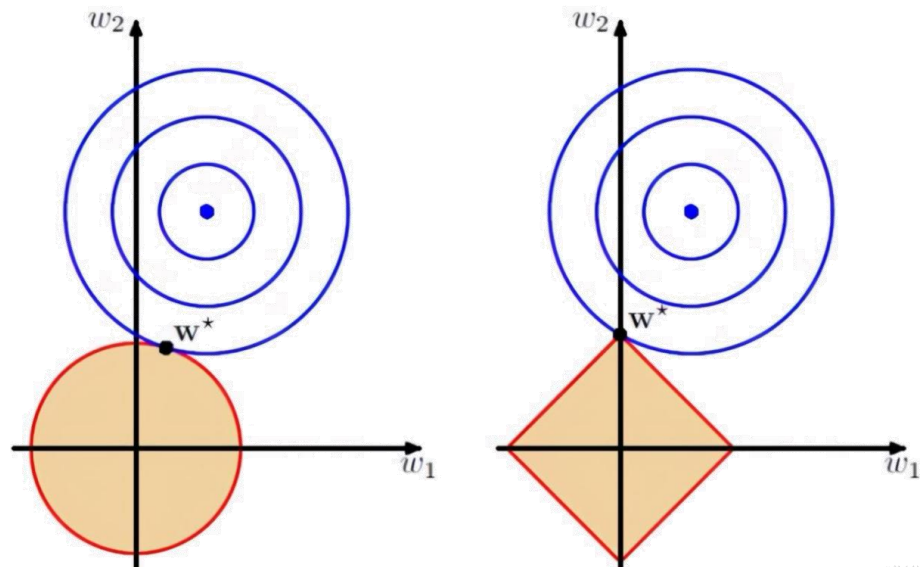
概率密度函数

$$= \operatorname{argmin}_{\mathbf{w}} \sum_{(x,y) \in \mathcal{D}} (y - \mathbf{w} \cdot \phi(x))^2$$

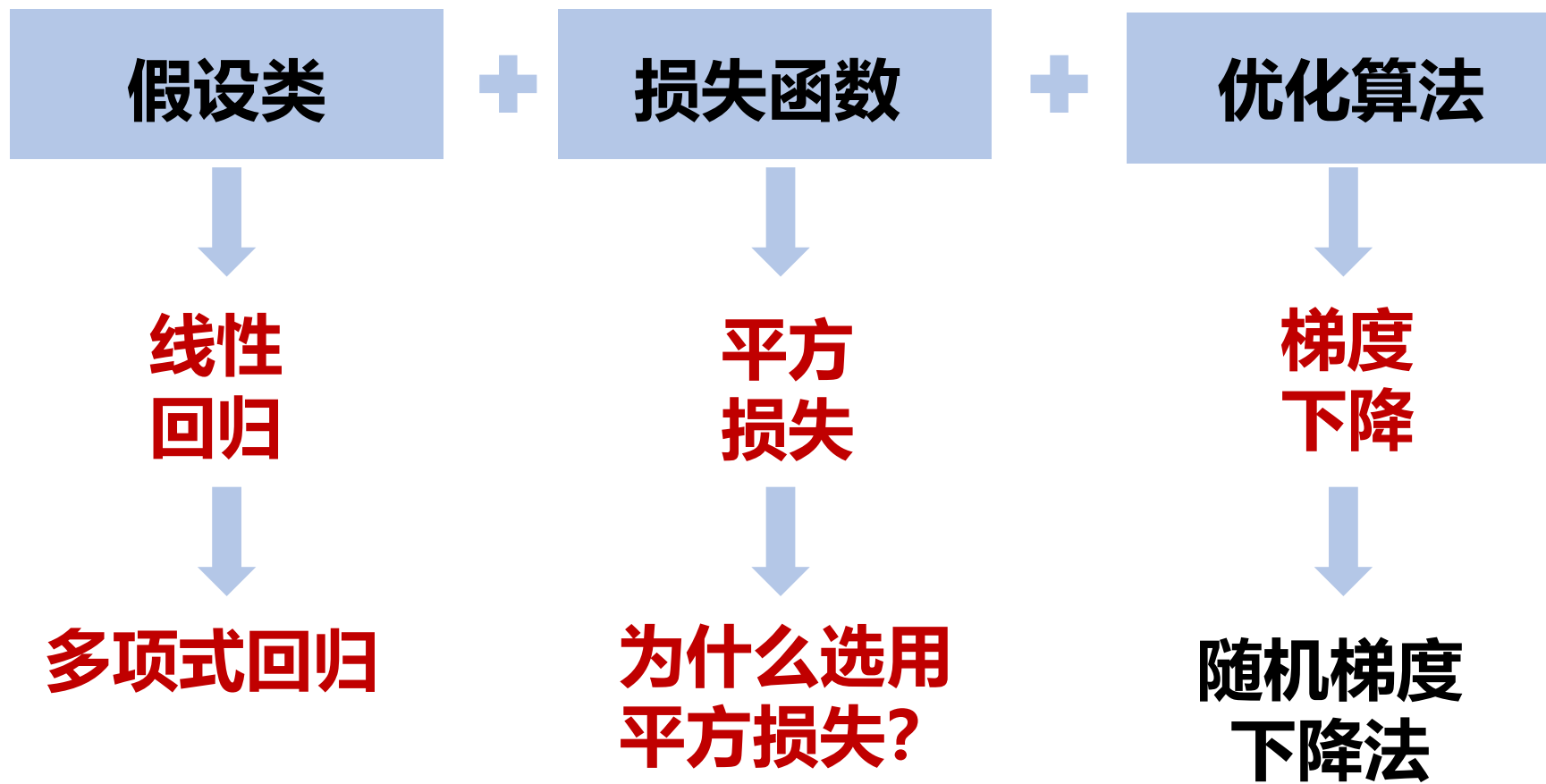
- 高斯模型下的最大似然估计 => 最小化平方损失

正则化

- 防止过拟合，提高模型的泛化能力
 - 不再仅仅最小化 $\sum_{(x,y) \in \mathcal{D}} (f_{\mathbf{w}}(x) - y)^2$;
 - 而是最小化 $\sum_{(x,y) \in \mathcal{D}} (f_{\mathbf{w}}(x) - y)^2 + \lambda R(\mathbf{w})$, $R(\mathbf{w})$ 被称为正则化项/惩罚项;
 - 常用的正则化项是 ℓ_1 正则化和 ℓ_2 正则化
 - ℓ_1 正则化: $R(\mathbf{w}) = \|\mathbf{w}\|_1$
 - ℓ_2 正则化: $R(\mathbf{w}) = \|\mathbf{w}\|_2^2$



线性回归：框架



回顾：凸/凹函数

- CONCAVE

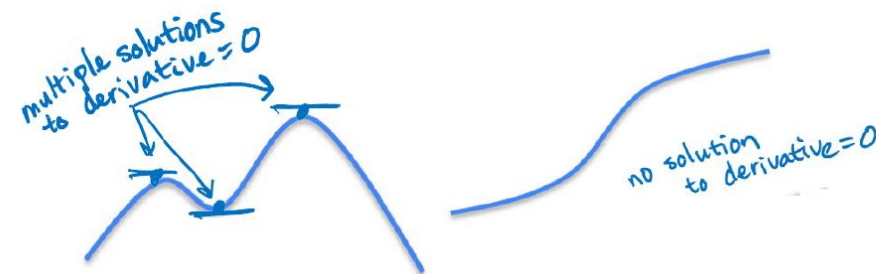
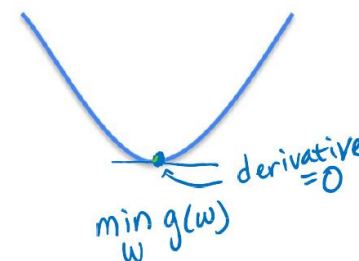
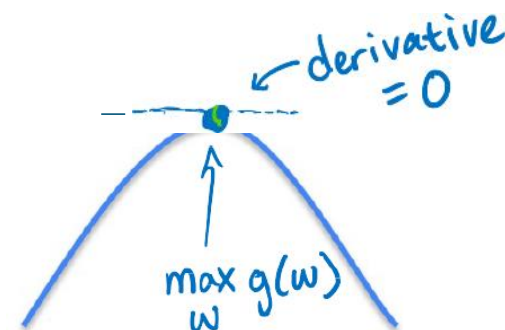
- 对于严格凹函数，若存在导数为零的点，则该点即为全局最大值点。

- CONVEX

- 对于严格凸函数，若存在导数为零的点，则该点即为全局最小值点。

- NEITHER

- 导数为0的点（若存在）无法预测是否为最大/小值点。



回顾：利用梯度上升找最大值

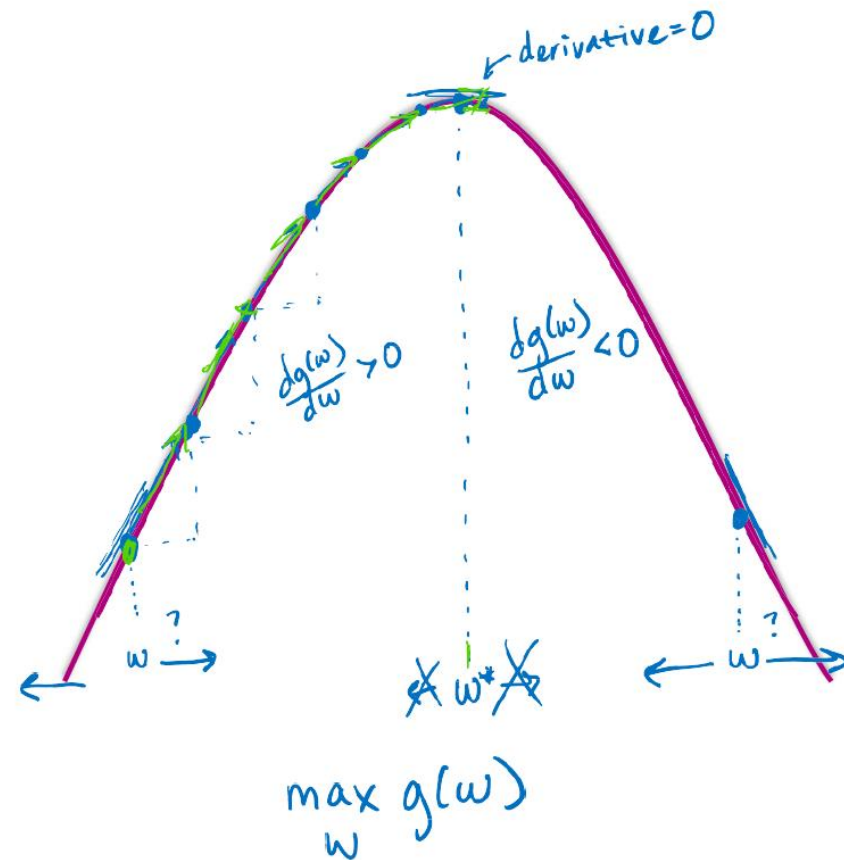
- 第 t 步的迭代，步长 (step size)

- 左侧 ($\frac{dg(w)}{dw} > 0$)

- $w^{(t+1)} = w^{(t)} + \eta \cdot \frac{dg(w)}{dw}$

- 右侧 ($\frac{dg(w)}{dw} < 0$)

- $w^{(t+1)} = w^{(t)} + \eta \cdot \frac{dg(w)}{dw}$



回顾：利用梯度下降找最小值

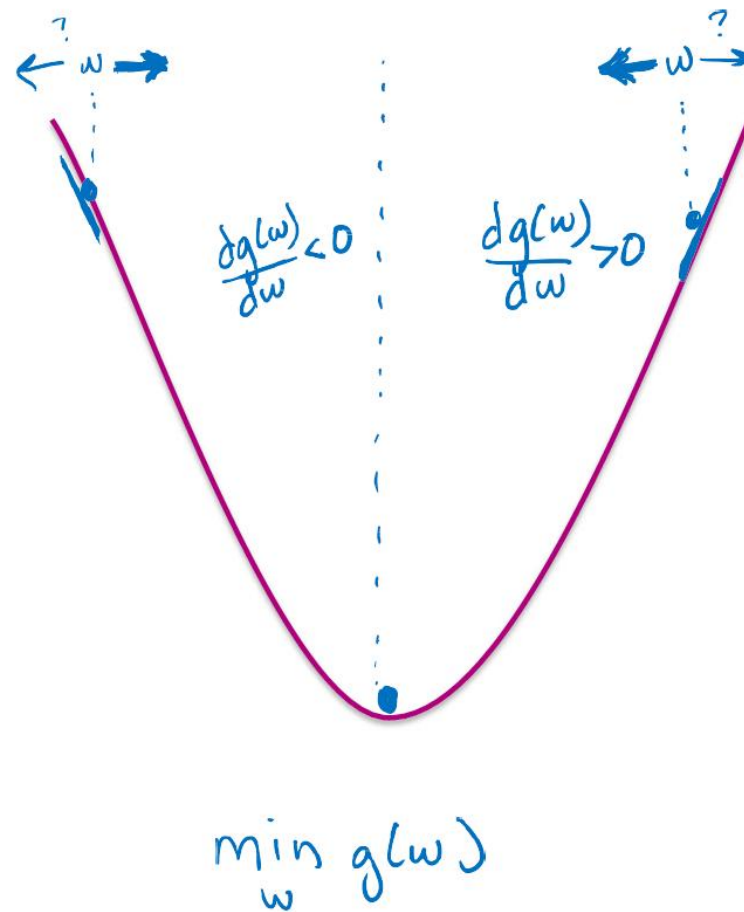
- 第 t 步的迭代，步长 (step size)

- 左侧 ($\frac{dg(\mathbf{w})}{d\mathbf{w}} < 0$)

- $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \cdot \frac{dg(\mathbf{w})}{d\mathbf{w}}$

- 右侧 ($\frac{dg(\mathbf{w})}{d\mathbf{w}} > 0$)

- $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \cdot \frac{dg(\mathbf{w})}{d\mathbf{w}}$



如何优化损失函数

- 求解模型参数的**目标**：最小化损失函数
- **梯度**：增加训练损失函数最快的方向

梯度下降法

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} \text{TrainLoss}(\mathbf{w})$$

η : step size : 步长/学习率

$\nabla_{\mathbf{w}} \text{TrainLoss}(\mathbf{w})$: 梯度

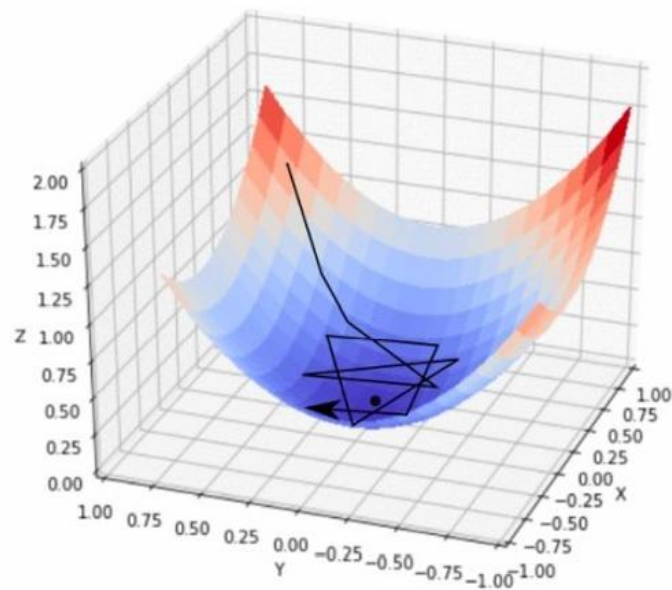


Algorithm: gradient descent

Initialize $\mathbf{w} = [0, \dots, 0]$

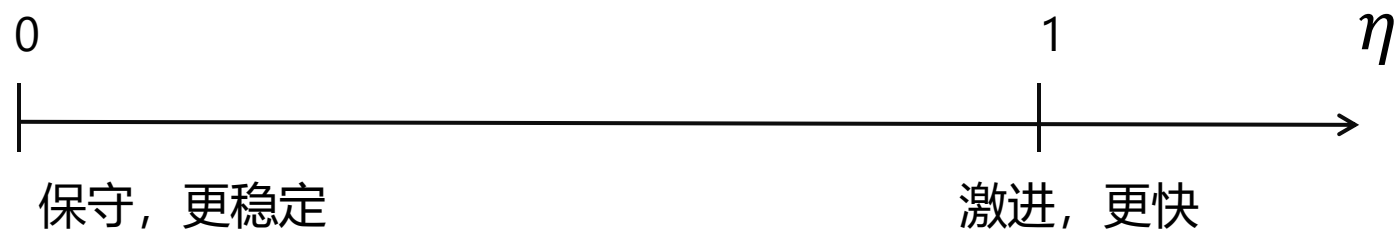
For $t = 1, \dots, T$: **epochs**

$$\mathbf{w} \leftarrow \mathbf{w} - \underbrace{\eta}_{\text{step size}} \underbrace{\nabla_{\mathbf{w}} \text{TrainLoss}(\mathbf{w})}_{\text{gradient}}$$



学习步长选择

- $\mathbf{w} \leftarrow \mathbf{w} - \eta \cdot \nabla_{\mathbf{w}} \text{TrainLoss}(x, y, \mathbf{w})$

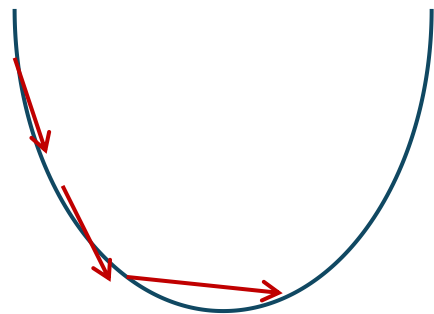


- 选择策略

- 守恒: $\eta = 0.1$
- 衰减: $\eta = 1/\sqrt{t}$
- 其他不错的策略:
 - 学习率衰减
 - Adagrad
 - Adam

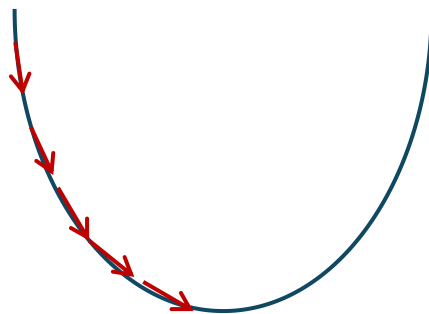
步长的计算

- 过大?
- 过小?



过大

当步长 η 过大，梯度下降可能会越过最小值。它可能无法收敛。

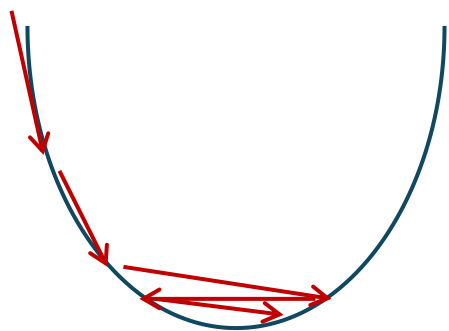


过小

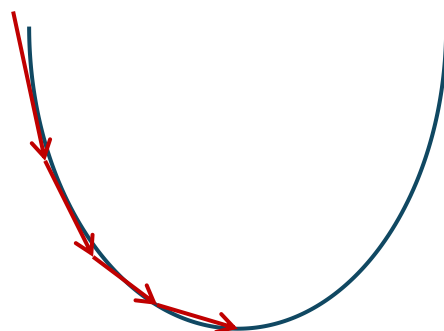
当步长 η 过小，梯度下降会很慢。

步长的计算

- 常用的策略：恒定 vs 递减步长

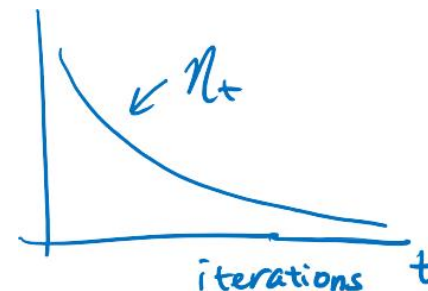


恒定



递减步长

$$\eta(t) = \frac{\alpha}{t}$$
$$\eta(t) = \frac{\alpha}{\sqrt{t}}$$

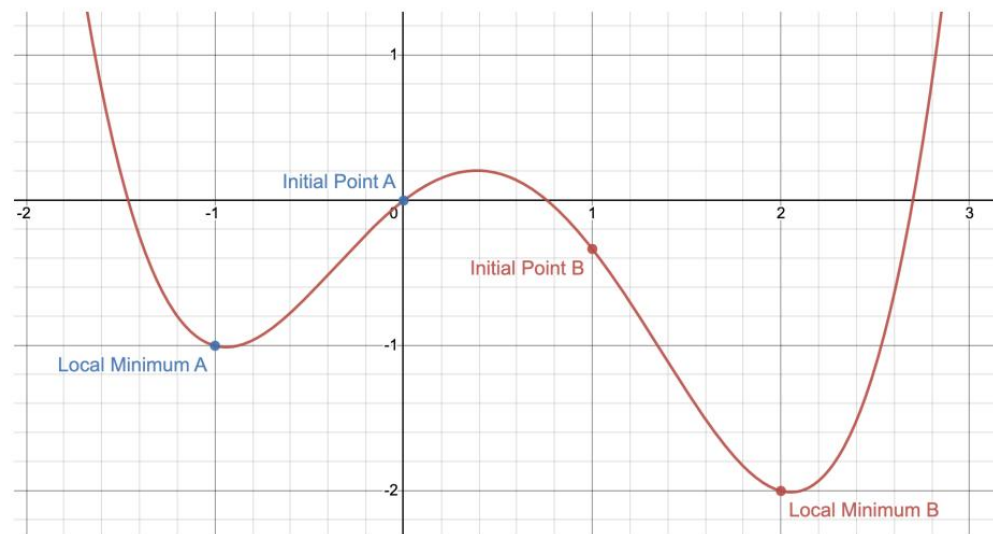
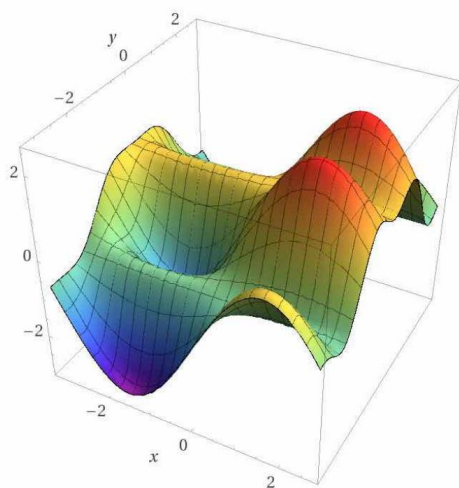
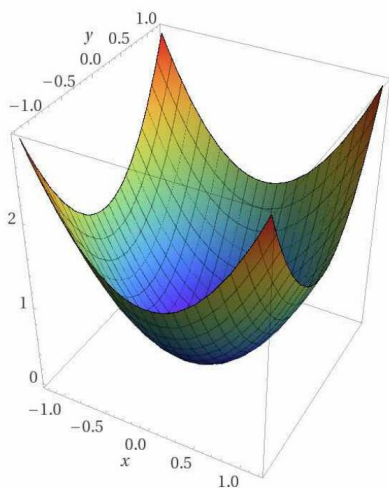


停止/收敛条件

- 对于凸函数，当梯度为零 ($\nabla_{\mathbf{w}} = 0$) 时收敛发生
- 在实际中，训练停止的条件：
 - $\|\nabla_{\mathbf{w}} \text{TrainLoss}(\mathbf{w})\| < \varepsilon$;
 - 达到最大迭代次数时停止;
 - 验证集上的性能不再提升
 - 即在验证集上监控指标（如验证损失、准确率等），如果连续若干轮没有改善，就停止训练.

非凸函数

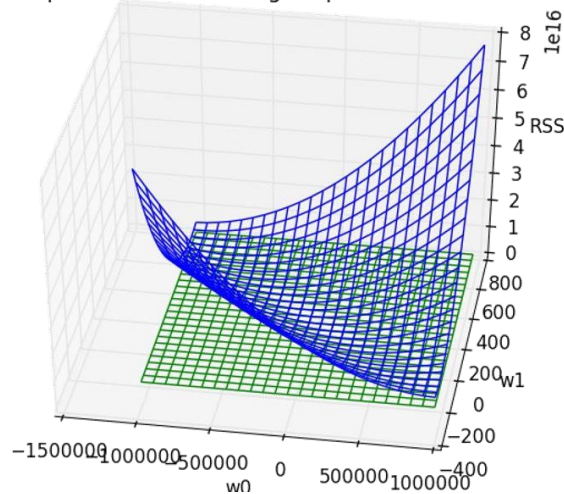
- 对于非凸函数可能会发生什么？
 - 损失函数有多个局部最小。梯度下降可能根据初始化收敛到不同局部最小。



多维梯度

$$\nabla_{\mathbf{w}} \text{TrainLoss}(\mathbf{w}) = \begin{bmatrix} \frac{d\text{TrainLoss}}{dw_1} \\ \frac{d\text{TrainLoss}}{dw_2} \\ \dots \\ \frac{d\text{TrainLoss}}{dw_d} \end{bmatrix}$$

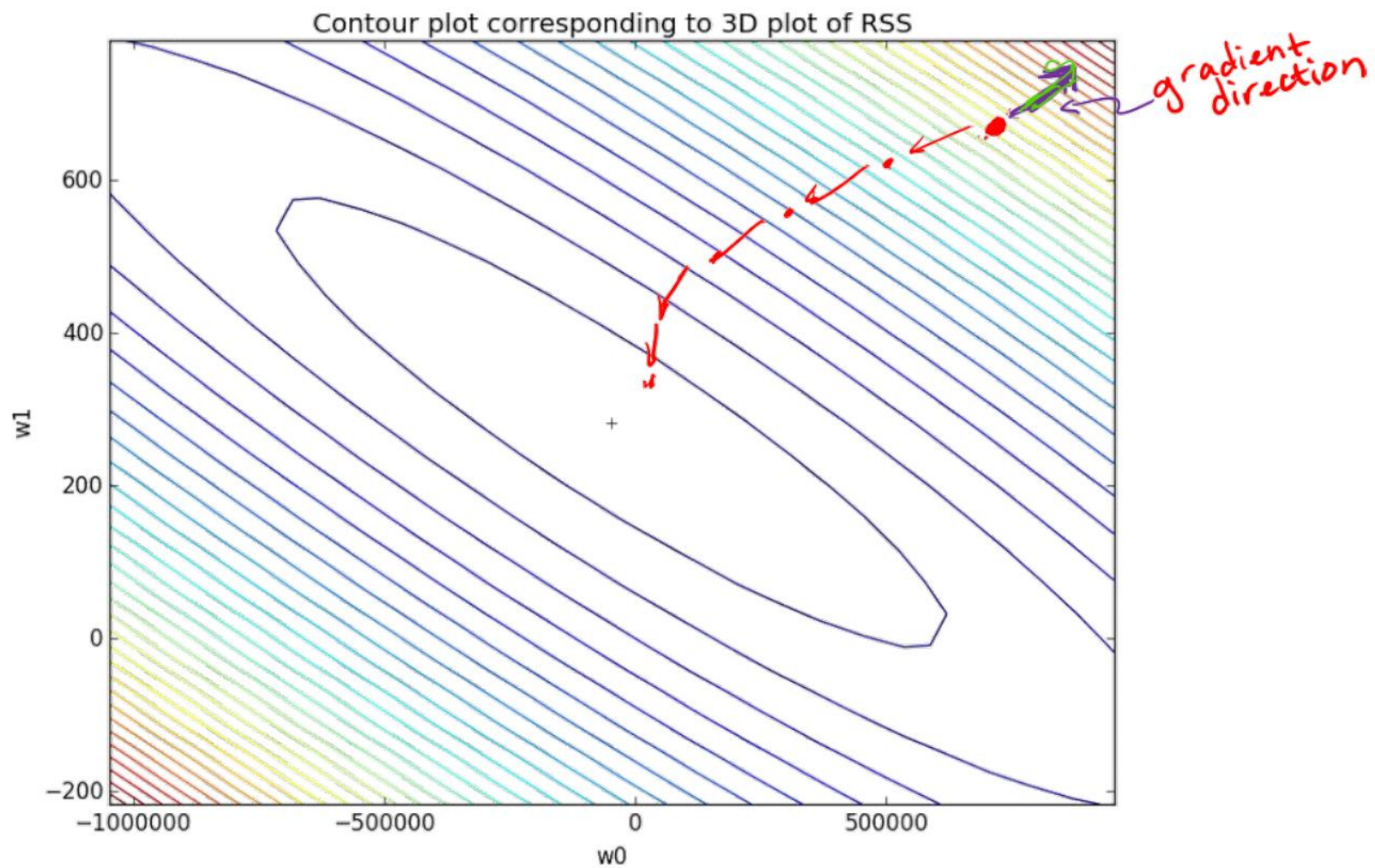
3D plot of RSS with tangent plane at minimum



- 梯度是一个向量，有其维度 $d \times 1$ ，其中 d 是特征的数量，包含偏置项。
- 特征提取器 $\phi(x) = [1, x]$
- 每个条目（即梯度向量的每个分量）都是仅针对单个 w_i 的偏导数，在求导时将其他所有变量均视为常数。
- 权重向量 $\mathbf{w} = [w_1, w_2]$

$$g(w) = 5w_1 + 10w_1w_2 + 2w_1^2,$$
$$\nabla g(w)?$$

多维梯度



线性回归的求解：梯度设置为0

- $$\begin{aligned}\underset{\mathbf{w}}{\operatorname{argmin}} \operatorname{TrainLoss}(\mathbf{w}) &= \underset{\mathbf{w}}{\operatorname{argmin}} \frac{1}{|\mathcal{D}_{\text{train}}|} (\mathbf{y} - \phi \mathbf{w})^T (\mathbf{y} - \phi \mathbf{w}) \\ &= \underset{\mathbf{w}}{\operatorname{argmin}} \underbrace{(\mathbf{y} - \Phi \mathbf{w})^T (\mathbf{y} - \Phi \mathbf{w})}_{\text{RSS}(\mathbf{w})} \quad \text{residual sum of squares (RSS)}\end{aligned}$$

- 对 $(\mathbf{y} - \Phi \mathbf{w})^T (\mathbf{y} - \Phi \mathbf{w})$ 求导：

$$\begin{aligned}\nabla_{\mathbf{w}} \text{RSS}(\mathbf{w}) &= \nabla_{\mathbf{w}} (\mathbf{y} - \Phi \mathbf{w})^T (\mathbf{y} - \Phi \mathbf{w}) \\ &= \nabla_{\mathbf{w}} (\mathbf{w}^T \Phi^T \Phi \mathbf{w} - \mathbf{w} \Phi^T \mathbf{y} - \mathbf{y}^T \Phi \mathbf{w}) \\ &= \nabla_{\mathbf{w}} (\mathbf{w}^T (\Phi^T \Phi) \mathbf{w} - 2 \mathbf{y}^T \Phi \mathbf{w}) \\ &= -2 \Phi^T (\mathbf{y} - \Phi \mathbf{w})\end{aligned}$$

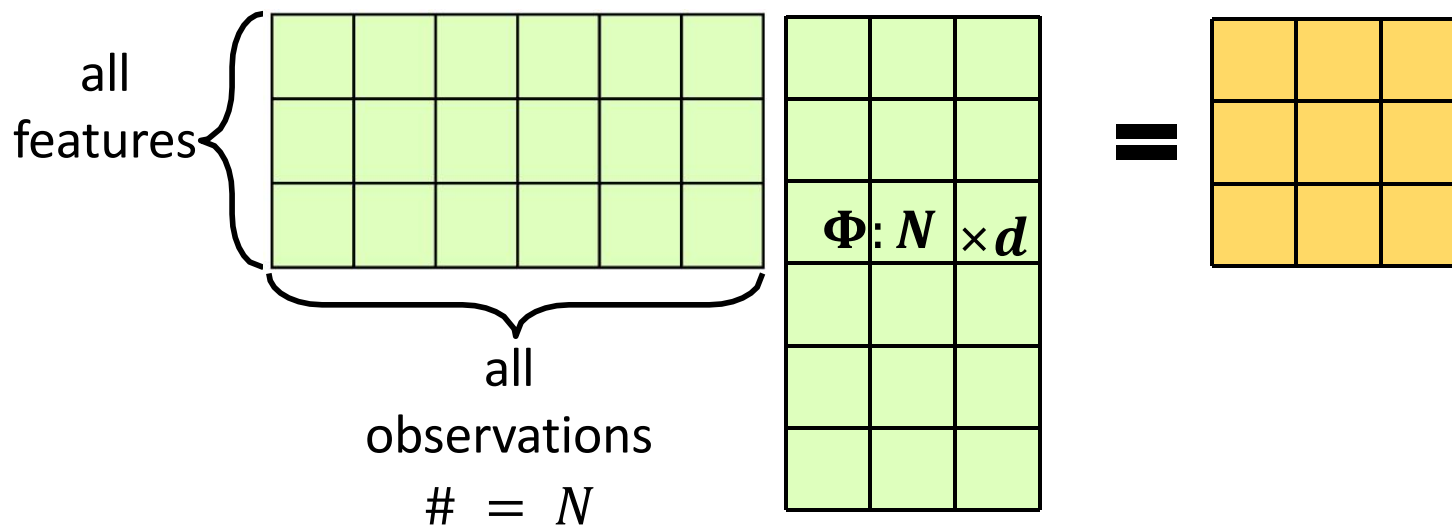
线性回归的求解：梯度设置为0

- 令导数为0，这里假设了 $\Phi^T \Phi$ 是可逆矩阵。

$$-2\Phi^T(\mathbf{y} - \Phi\mathbf{w}) = 0$$

$$\mathbf{w} = (\Phi^T \Phi)^{-1} \Phi^T \mathbf{y}$$

$\Phi^T \Phi$ 不可逆怎么办？



线性回归的求解：梯度下降

- 目标损失函数：

$$\text{TrainLoss}(\mathbf{w}) = \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{(x,y) \in \mathcal{D}_{\text{train}}} (f_{\mathbf{w}}(x) - y)^2$$

- 使用链式法则：

$$\nabla_{\mathbf{w}} \text{TrainLoss}(\mathbf{w}) = \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{(x,y) \in \mathcal{D}_{\text{train}}} 2(\mathbf{w} \cdot \phi(x) - y) \phi(x)$$

线性回归的求解：梯度下降

- 一个例子

training data $\mathcal{D}_{\text{train}}$

x	y
1	1
2	3
4	3

$$\nabla_{\mathbf{w}} \text{TrainLoss}(\mathbf{w}) = \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{(x,y) \in \mathcal{D}_{\text{train}}} 2(\mathbf{w} \cdot \phi(x) - y) \phi(x)$$

梯度更新 $\eta = 0.1$:

$$\mathbf{w} \leftarrow \mathbf{w} - 0.1 \cdot \nabla_{\mathbf{w}} \text{TrainLoss}(\mathbf{w})$$

t	$\nabla_{\mathbf{w}} \text{TrainLoss}(\mathbf{w})$	$\mathbf{w}$
		$[0, 0]$
1	$\frac{1}{3} (2([0, 0] \cdot [1, 1] - 1)[1, 1] + 2([0, 0] \cdot [1, 2] - 3)[1, 2] + 2([0, 0] \cdot [1, 4] - 3)[1, 4])$ $= [-4.67, -12.67]$	$[0.47, 1.27]$
2	$\frac{1}{3} (2([0.47, 1.27] \cdot [1, 1] - 1)[1, 1] + 2([0.47, 1.27] \cdot [1, 2] - 3)[1, 2] + 2([0.47, 1.27] \cdot [1, 4] - 3)[1, 4])$ $= [2.18, 7.24]$	$[0.25, 0.54]$
...	...	...
200	$\frac{1}{3} (2([1, 0.57] \cdot [1, 1] - 1)[1, 1] + 2([1, 0.57] \cdot [1, 2] - 3)[1, 2] + 2([1, 0.57] \cdot [1, 4] - 3)[1, 4])$ $= [0, 0]$	$[1, 0.57]$

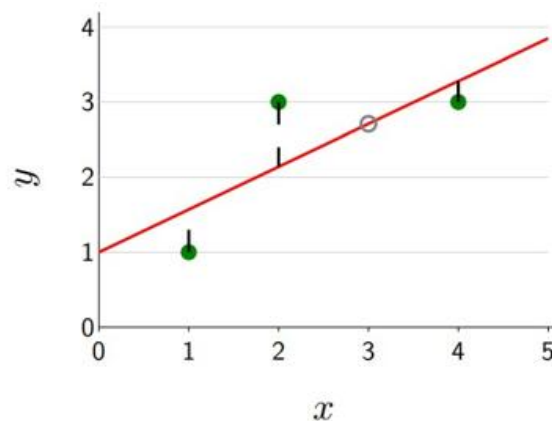
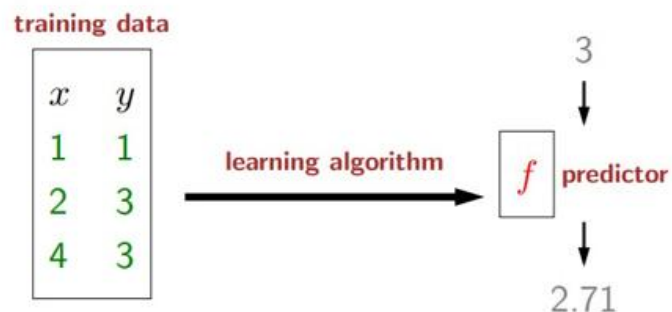
线性回归：总结

- 哪些预测器是可行的？
 - 假设类
- 如何衡量预测器好坏？
 - 损失函数
- 如何计算出最好的预测器？
 - 优化算法

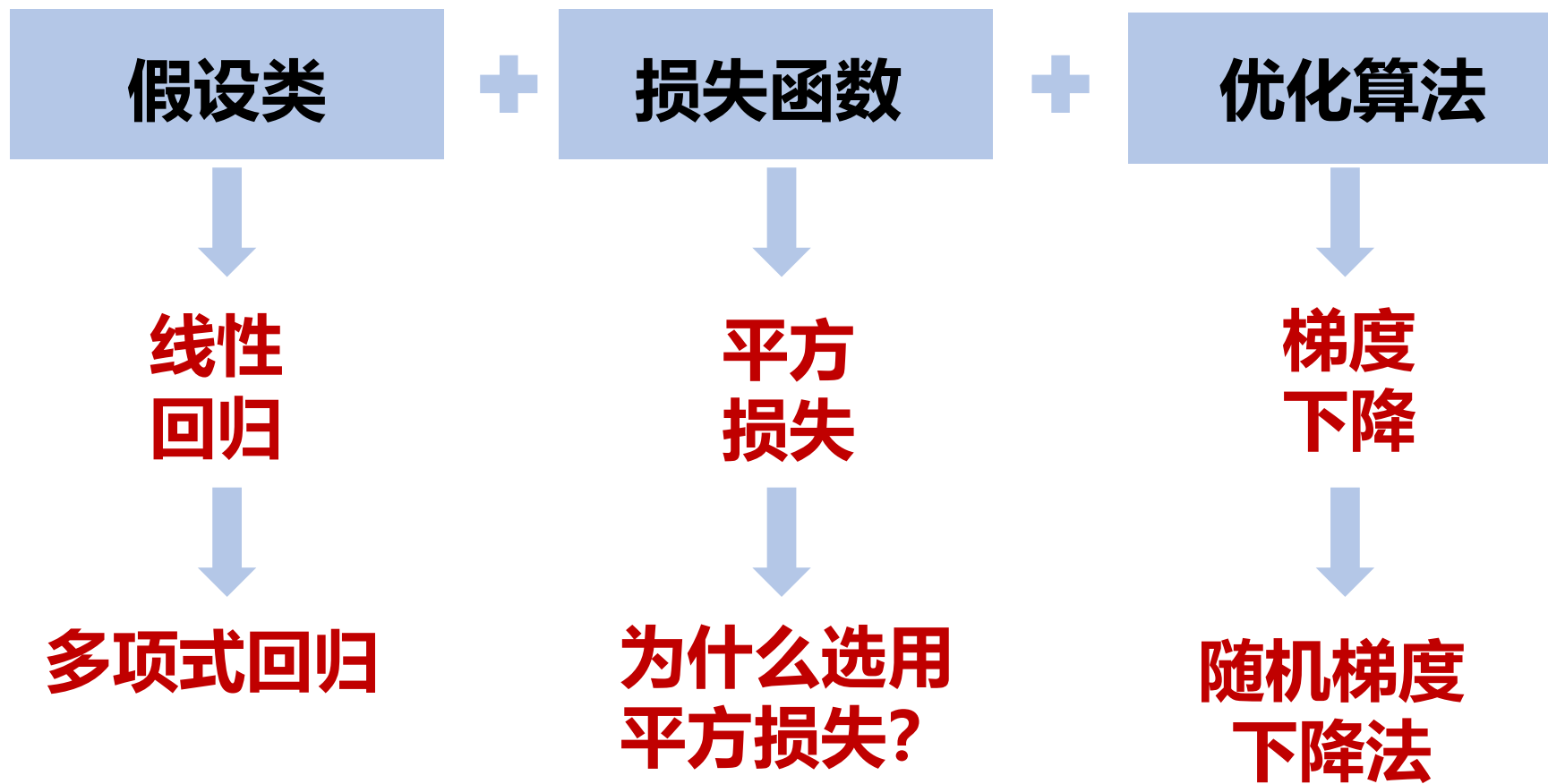
$$\mathcal{F} = \{f_{\mathbf{w}}(x) = \mathbf{w} \cdot \phi(x)\}, \phi(x) = [1, x]$$

$$\text{Loss}(x, y, \mathbf{w}) = (f_{\mathbf{w}}(x) - y)^2$$

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla \text{TrainLoss}(\mathbf{w})$$

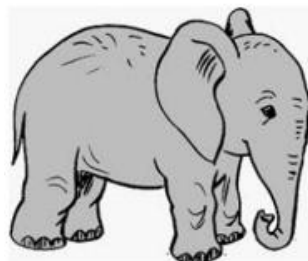


线性回归：框架



梯度下降的缺点

- $\text{TrainLoss}(\mathbf{w}) = \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{(x,y) \in \mathcal{D}_{\text{train}}} (f_{\mathbf{w}}(x) - y)^2$



Algorithm: gradient descent

Initialize $\mathbf{w} = [0, \dots, 0]$

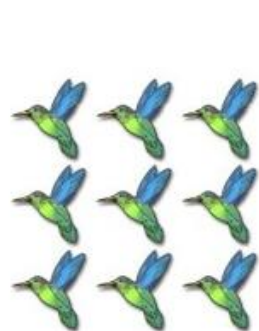
For $t = 1, \dots, T$:

$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} \text{TrainLoss}(\mathbf{w})$

- 批量梯度下降在每走一步之前都需要扫描整个训练集
- 当训练量 N 很大时，遍历所有样本是非常耗费资源、非常耗时的

随机梯度下降

- 随机梯度下降 (Stochastic Gradient Descent, SGD) : 仅根据单个训练样本的误差梯度来更新参数。



Algorithm: stochastic gradient descent

Initialize $\mathbf{w} = [0, \dots, 0]$

For $t = 1, \dots, T$:

For $(x, y) \in \mathcal{D}_{\text{train}}$:

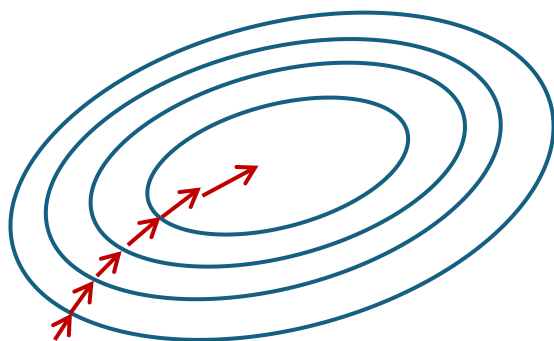
$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} \text{Loss}(x, y, \mathbf{w})$

$$\text{TrainLoss}(\mathbf{w}) = \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{(x, y) \in \mathcal{D}_{\text{train}}} \text{Loss}(x, y, \mathbf{w})$$

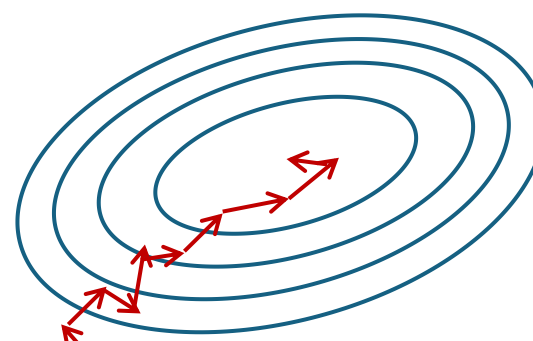
- 小批量随机梯度下降 (Minibatch SGD) : 每次更新由 B 个样本的平均梯度计算得出。

梯度下降 vs 随机梯度下降

- 可视化



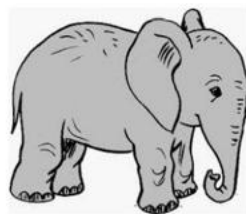
梯度下降



随机梯度下降

总结

- $\text{TrainLoss}(\mathbf{w}) = \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{(x,y) \in \mathcal{D}_{\text{train}}} (f_{\mathbf{w}}(x) - y)^2$



gradient descent



stochastic gradient descent

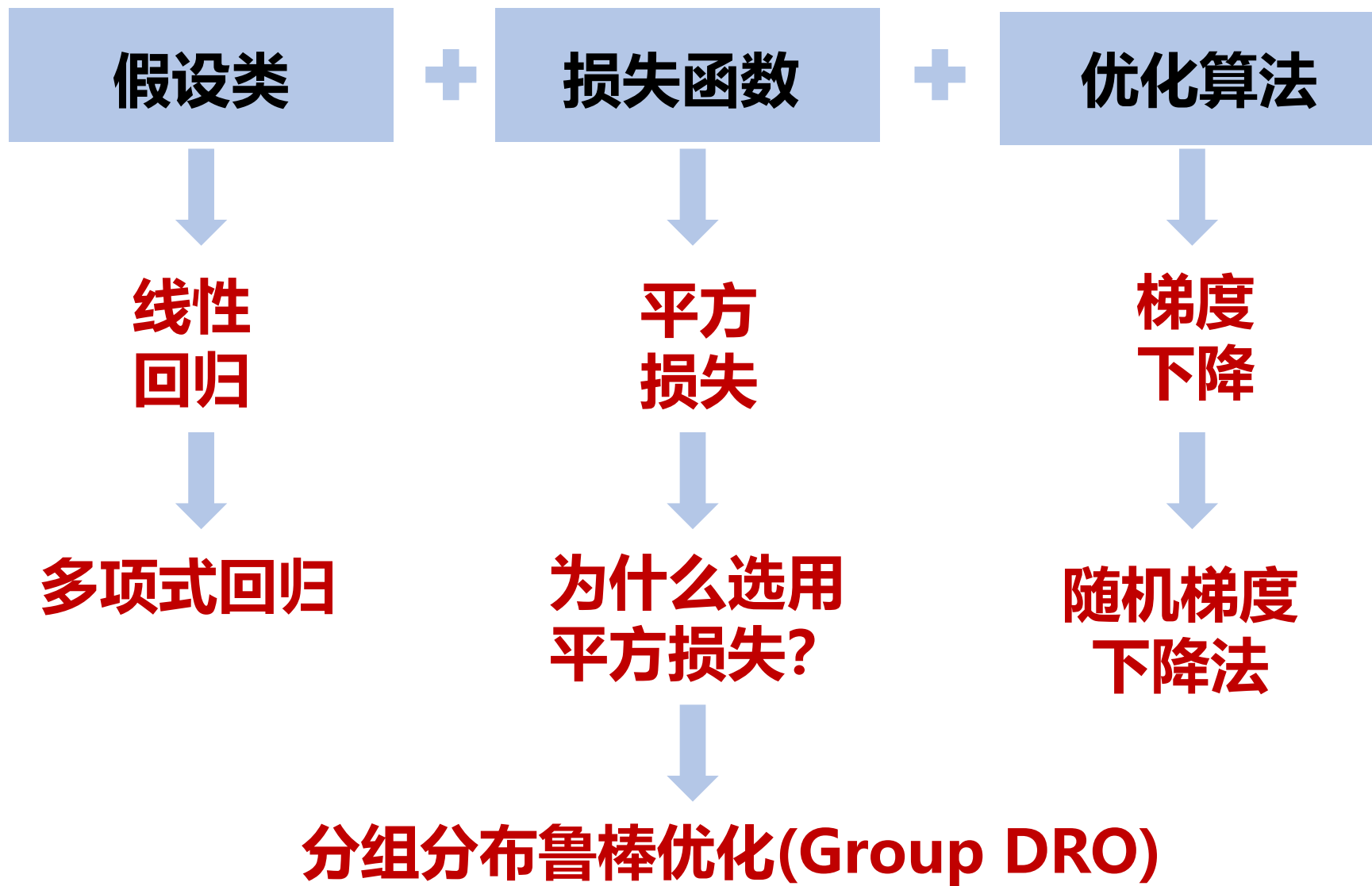


Key idea: stochastic updates

It's not about **quality**, it's about **quantity**.

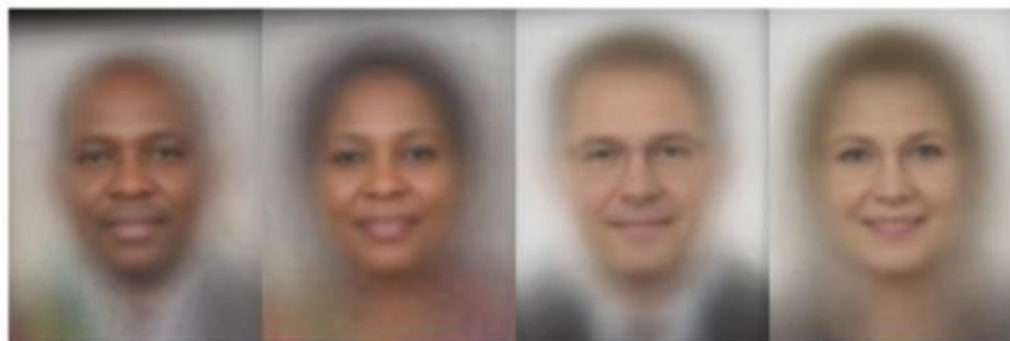
- SGD每次用的梯度虽然不准（质量低）
- 但它更新次数多（数量多），反而能更快地收敛

线性回归：框架



关于性别的不公平性

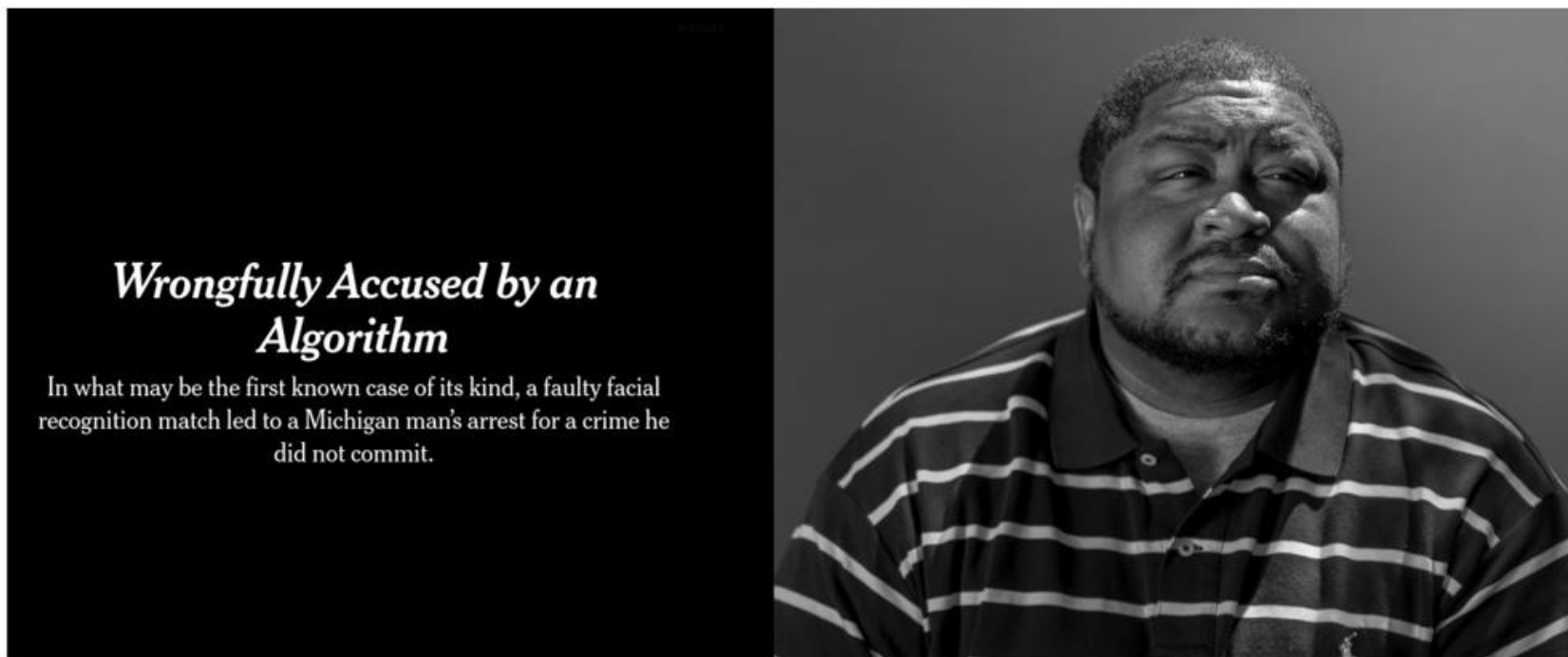
Gender Classifier	Darker Male	Darker Female	Lighter Male	Lighter Female	Largest Gap
 Microsoft	94.0% 	79.2% 	100% 	98.3% 	20.8% 
 FACE++	99.3% 	65.5% 	99.2% 	94.0% 	33.8% 
 IBM	88.0% 	65.3% 	99.7% 	92.9% 	34.4% 



机器学习中存在不平等性

人脸识别造成的冤案

- 警方调查2018年一起手表店盗窃案时，将监控中模糊的嫌疑人图像输入了面部识别系统。
- 仅凭借一张驾照照片锁定了42岁的Robert Williams。



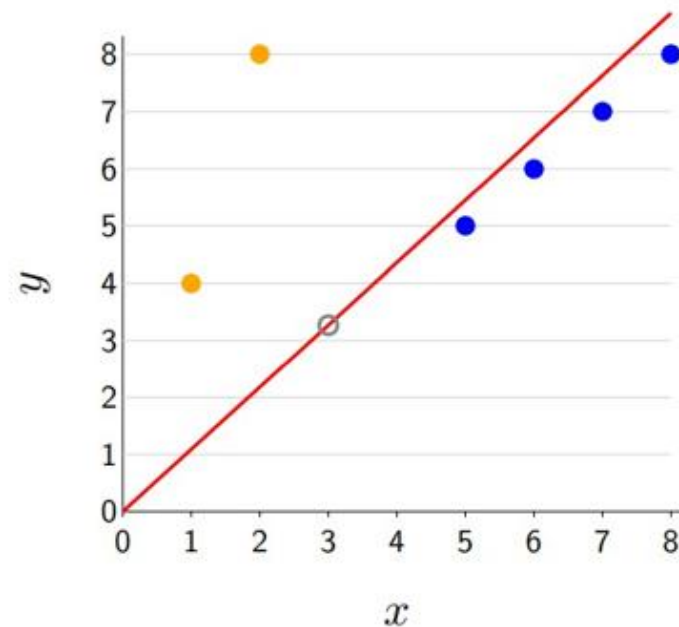
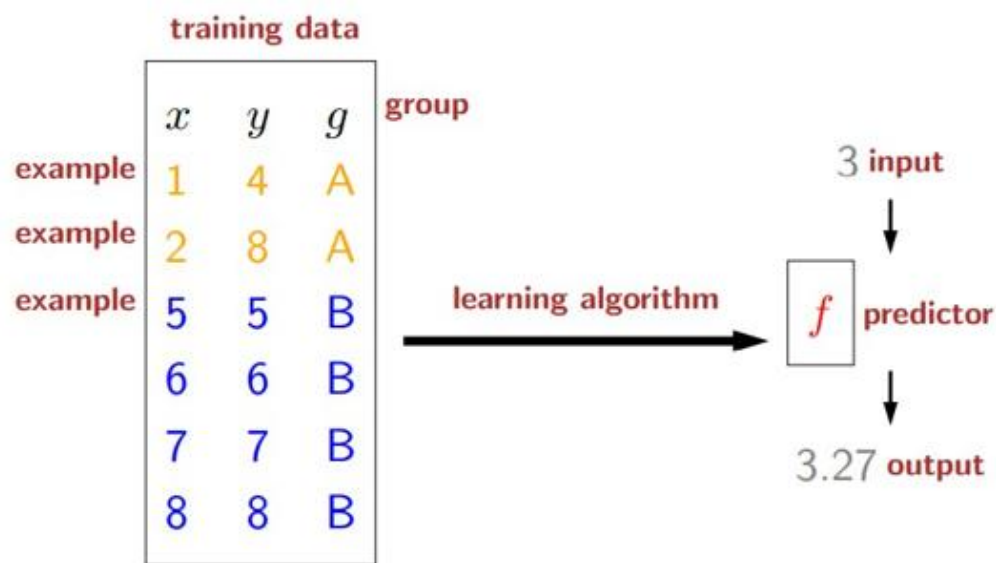
相似案件不止一起

- 在新泽西州，警方在用人脸识别逮捕一位名叫Nijeer Parks的黑人男子后，将他在监狱关了10天，才发现抓错了人。



线性回归在不同分组上的效果

- 尽管模型没有使用组别信息，但不公平性存在！

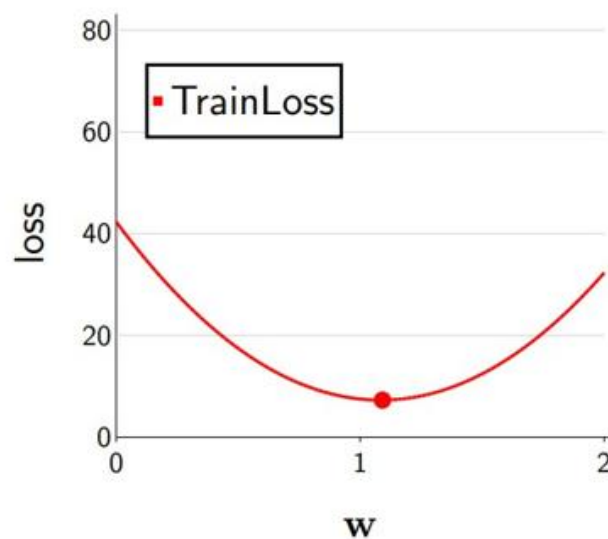


$$f_{\mathbf{w}}(x) = \mathbf{w} \cdot \phi(x) \quad \mathbf{w} = [\omega] \quad \phi(x) = [x]$$

平均损失

- $\text{TrainLoss}(\mathbf{w}) = \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{(x,y) \in \mathcal{D}_{\text{train}}} (f_{\mathbf{w}}(x) - y)^2$

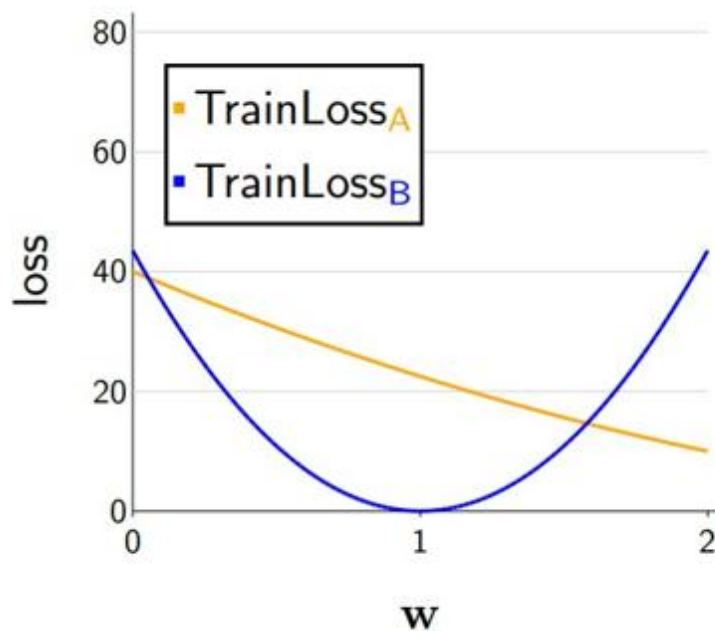
x	y	g
1	4	A
2	8	A
5	5	B
6	6	B
7	7	B
8	8	B



分组计算损失

- $\text{TrainLoss}_g(\mathbf{w}) = \frac{1}{|\mathcal{D}_{\text{train}}(g)|} \sum_{(x,y) \in \mathcal{D}_{\text{train}}(g)} (f_{\mathbf{w}}(x) - y)^2$
 - $\text{TrainLoss}_A(1) = \frac{1}{2} ((1 - 4)^2 + (2 - 8)^2) = 22.5$
 - $\text{TrainLoss}_B(1) = \frac{1}{4} ((5 - 5)^2 + (6 - 6)^2 + (7 - 7)^2 + (8 - 8)^2) = 0$

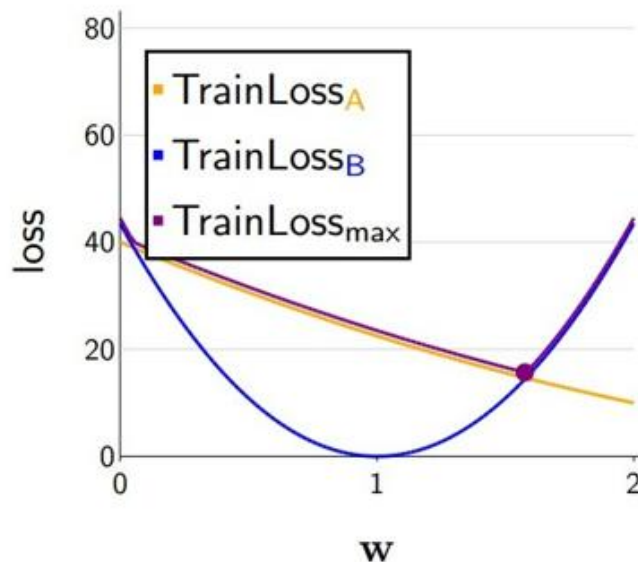
x	y	g
1	4	A
2	8	A
5	5	B
6	6	B
7	7	B
8	8	B



计算组内损失最大值

- $\text{TrainLoss}_{\max}(\mathbf{w}) = \max_g \text{TrainLoss}_g(\mathbf{w})$
 - $\text{TrainLoss}_A(1) = 22.5$
 - $\text{TrainLoss}_B(1) = 0$
 - $\text{TrainLoss}_{\max}(\mathbf{w}) = \max(22.5, 0) = 22.5$

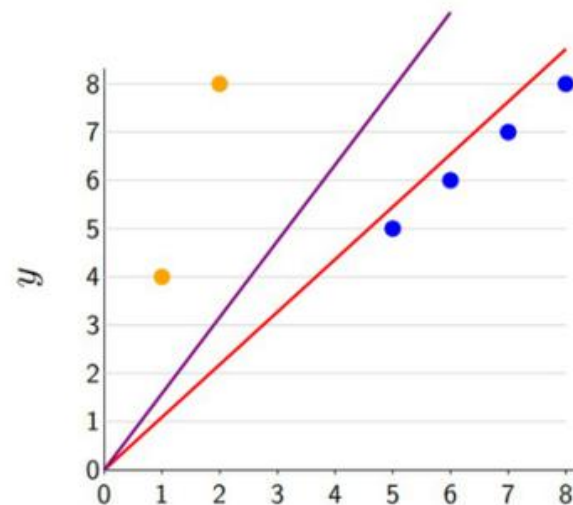
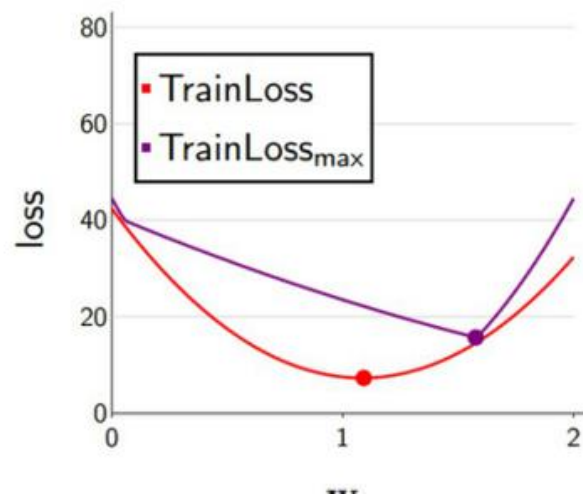
x	y	g
1	4	A
2	8	A
5	5	B
6	6	B
7	7	B
8	8	B



平均损失 vs 最大组内损失

- 标准优化：
 - 最小化平均损失: $w = 1.09$
- 分组分布鲁棒优化：
 - 最小化最大组损失: $w = 1.58$

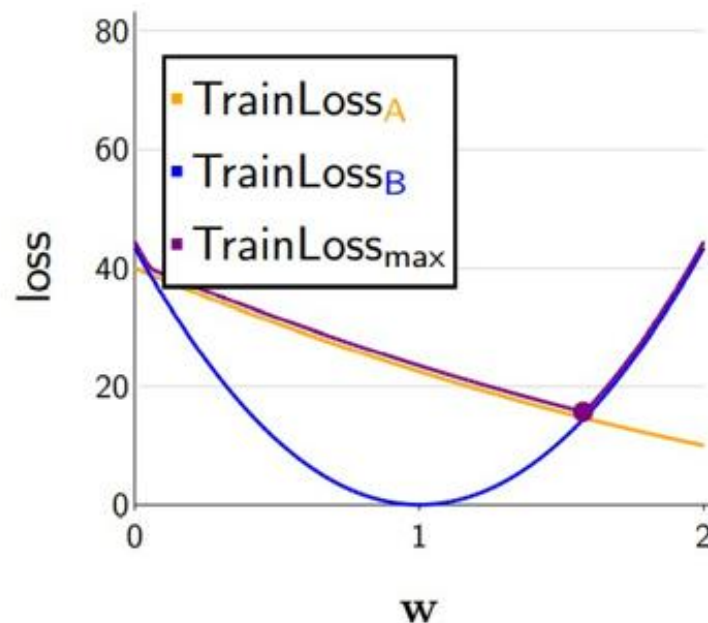
x	y	g
1	4	A
2	8	A
5	5	B
6	6	B
7	7	B
8	8	B



通过梯度下降法优化

- $\text{TrainLoss}_{\max}(\mathbf{w}) = \max_g \text{TrainLoss}_g(\mathbf{w})$
- $\nabla \text{TrainLoss}_{\max}(\mathbf{w}) = \nabla \text{TrainLoss}_{g^*}(\mathbf{w})$
- $g^* = \operatorname{argmax}_g \text{TrainLoss}_g(\mathbf{w})$

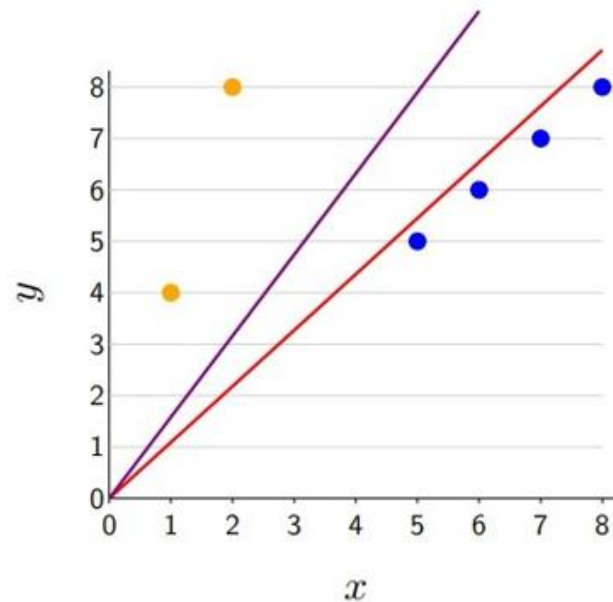
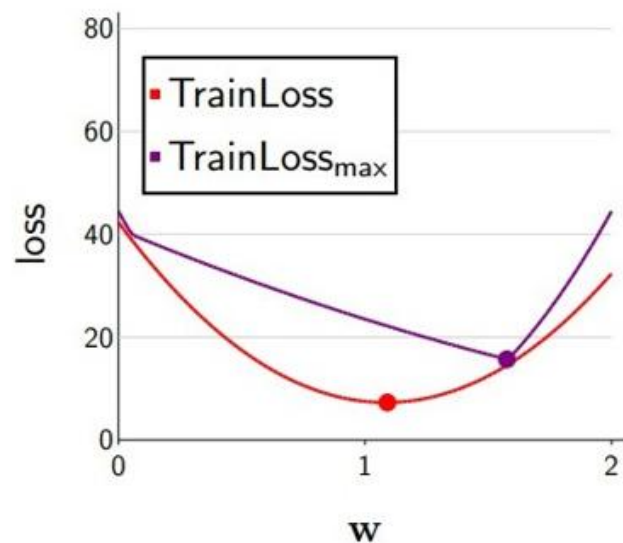
x	y	g
1	4	A
2	8	A
5	5	B
6	6	B
7	7	B
8	8	B



总结

- 最大组损失 $\neq$ 平均损失
- 分组分布鲁棒优化：最小化最大组损失

x	y	g
1	4	A
2	8	A
5	5	B
6	6	B
7	7	B
8	8	B



Questions

